Temco Platform

TEMCO Platform — Engineering Design (Revision 2) · built Nov 12, 2026

TEMCO Platform — Engineering Design (Revision 2)

Host: the VS Code fork. UI library: Fluent UI v9. Editor engine: ONLYOFFICE Docs.

Status: design for implementation. Written to be handed to a code generator. Target repository: `temcocontrols/temco-platform` (VS Code 1.144.0.0 fork, 12 `temco-*` extensions).

0. How to use this document

0.1 Revision history — what changed, and why it is not cosmetic

Revision 1 proposed a standalone web app (React 19 + Vite) and pushed the VS Code fork down to "an adapter". Revision 2 inverts that. The product **is** the fork; the workbench is the shell users already know; and every surface we own is built with Fluent UI v9 inside it. Three reversals:

#	Revision 1	Revision 2	Reason
1	Host = new web app	Host = the existing VS Code fork	Discards a working shell, keybindings, editor model, file watching, and the installed base for nothing
2	Editor = first-party Tiptap for most formats	Editor = ONLYOFFICE Docs for Office formats	Office fidelity is the actual requirement; re-implementing DOCX layout is not a business we are in
3	UI library = free choice	Fluent UI v9 is mandatory in the content register, forbidden in the shell register	One visual language for our screens, zero risk to the shell

Everything else — the five core concepts, the domain model, the publication state machine, the flows, the round-trip law — survives. Only where it is *hosted* changed.

0.2 Reading order for a code generator

Read in order. Do not generate UI before §2 (the two-register law) and §4 (the document server contract), because those two sections decide which library owns which pixel and which process owns which file.

0.3 Five rules that override anything else in this document

1. **The shell is not ours.** Never restyle, re-implement, or replace the title bar, activity bar, side bar trees, tabs, status bar, command palette, or settings editor. They are VS Code. They use VS Code theme variables. They do not import Fluent.
2. **Fluent does not leak.** Fluent UI v9 is imported only inside webview content (§2.2). No Fluent component is rendered into shell DOM, and no Fluent token is used to colour a shell surface.
3. **The document server is not a library.** ONLYOFFICE is a separate HTTP service. Code must treat it as an unreliable remote dependency with a health state, not as an importable editor.
4. **A save is a round trip through the storage contract.** No editor writes to the asset store directly. Every save is `editor → callback → host validates → store writes new revision`. (§4.5)
5. **Do not invent a domain concept.** Products, Assets, Editor Hosts, Publications, Knowledge. If a screen needs a sixth noun, the design is wrong — stop and re-read §5.

---

1. Vision and principles

TEMCO Platform is an engineering operating system: it holds documentation, hardware descriptions, firmware, programs, releases, and the knowledge derived from them, for products built and supported for ten or more years.

The user is a controls engineer at 2 a.m. with a T3000 that will not talk to a BACnet router. The product succeeds when that engineer finds the right page, sees the firmware version it applies to, and gets a file they can open.

1.1 Principles and what each one forbids in code

Principle	Forbids
Familiar first	A custom shell, custom tabs, a custom command palette, custom keybindings
One editor column	Side-by-side editor panes; split editors for the same asset
Structure is data	Hard-coded folder names, hard-coded product lists, "T3000" in a component
Every save is a revision	In-place file writes, "Save" without a version
Degrade loudly	A blank iframe, a silent spinner, a swallowed error
The tree is the map	A screen that shows assets but cannot be reached from the tree

1.2 Success conditions (measure these, not vibes)

- Open a 40-page `.docx` from the tree to a usable editor in **under 4 s warm**, **under 9 s cold**.
- The tree never shows a spinner for more than **300 ms** on a warm start.

- A Markdown file opened and closed without an edit produces **zero** revisions and a clean tab.
- Every asset type in §5.3 has exactly one surface that opens it, and the mapping is table-driven.
- With the document server down, every Office asset still opens **read-only with a stated reason** — never blank.

---

2. The two-register law

This is the single most important structural decision in the document. The workbench and our content are two different UI stacks, and confusing them is how this project fails.

2.1 The boundary

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ REGISTER A — THE SHELL                          owner: VS Code core      │
│ library: none (native workbench DOM + CSS)                               │
│ styling: --vscode-* theme variables only                                 │
│                                                                          │
│   title bar · activity bar · side bar trees · tabs · breadcrumbs ·       │
│   status bar · command palette · notifications · quick pick · settings   │
├──────────────────────────────────────────────────────────────────────────┤
│ REGISTER B — THE CONTENT                        owner: temco extensions  │
│ library: Fluent UI v9 + React 19                                         │
│ styling: Fluent tokens, bridged from --vscode-* (§7.2)                   │
│                                                                          │
│   every custom editor webview · every webview view panel ·               │
│   every dialog drawn inside our own surfaces                             │
├──────────────────────────────────────────────────────────────────────────┤
│ REGISTER C — THE REMOTE ENGINE                  owner: ONLYOFFICE Docs   │
│ library: ONLYOFFICE's own web app                                        │
│ styling: ONLYOFFICE's own themes, told to match ours                     │
│                                                                          │
│   the document canvas inside an <iframe>, in register B's webview only   │
└──────────────────────────────────────────────────────────────────────────┘
```

2.2 What each register may use

	Shell (A)	Content (B)	Remote (C)
Fluent UI v9	**Never**	**Always**	n/a
React	Never	Always	n/a
VS Code `TreeView`	Yes	**Never**	n/a
VS Code theme vars	Yes	Only read, to feed §7.2	n/a
Own context menu	Only via `menus` contributions	Yes (Fluent `Menu`)	Own menus
Own keyboard shortcuts	Only via `keybindings` contributions	Only non-chorded, scoped keys	Own, inside its iframe

2.3 Why Fluent cannot be "the main lib for VS Code" — and what we do instead

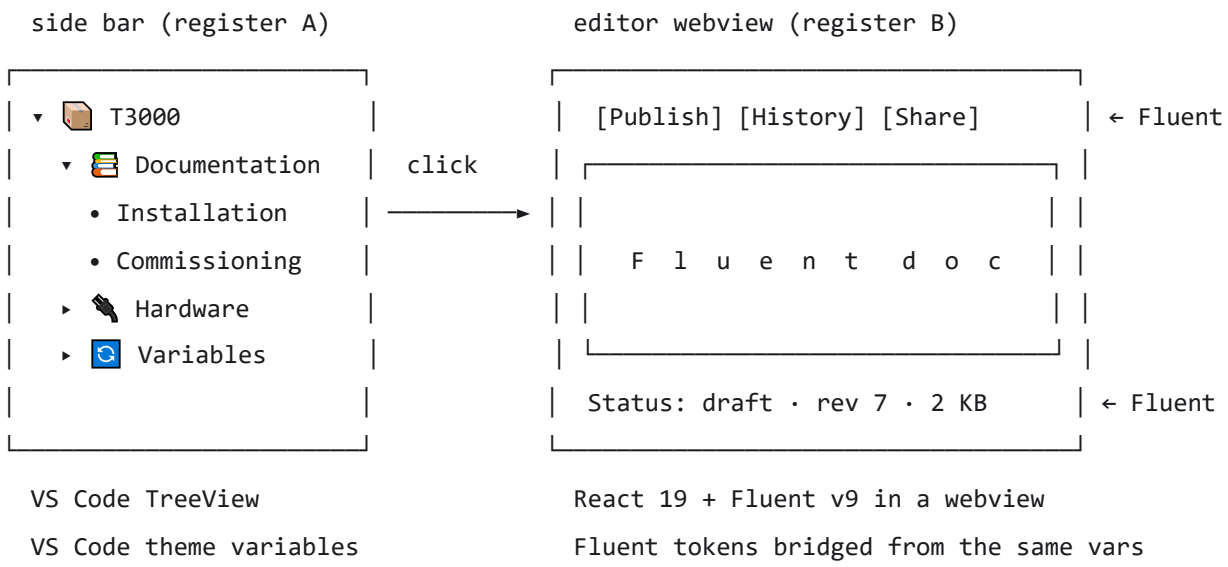
VS Code's workbench is hand-written DOM driven by DI, not React. Replacing its activity bar with `FluentProvider` does not produce a Fluent activity bar; it produces a broken one, and it breaks every extension that positions against those parts. What we can honestly do, and what this design does:

1. **Every pixel we draw is Fluent.** All our screens live in webviews, and in webviews Fluent is the only component library. Our surfaces will look and behave like one product because they share one library and one theme bridge.
2. **The bridge makes Fluent look native.** §7.2 reads VS Code's live theme and rewrites Fluent's tokens, so a Fluent `Button` in our editor panel is the same colour as a VS Code button beside it. This is what makes the seam invisible.
3. **Where a native control already exists, we use it through the host.** Adding an asset is a `TreeView` with `view/item/context` menu contributions, not a Fluent list in a webview. This is faster, accessible for free, and themable for free.

Rule of thumb: if the control is *chrome* (navigation, tabs, trees, commands), it is VS Code. If the control is *content* (a form, a table of releases, a publish checklist, a document), it is Fluent.

2.4 The seam, drawn

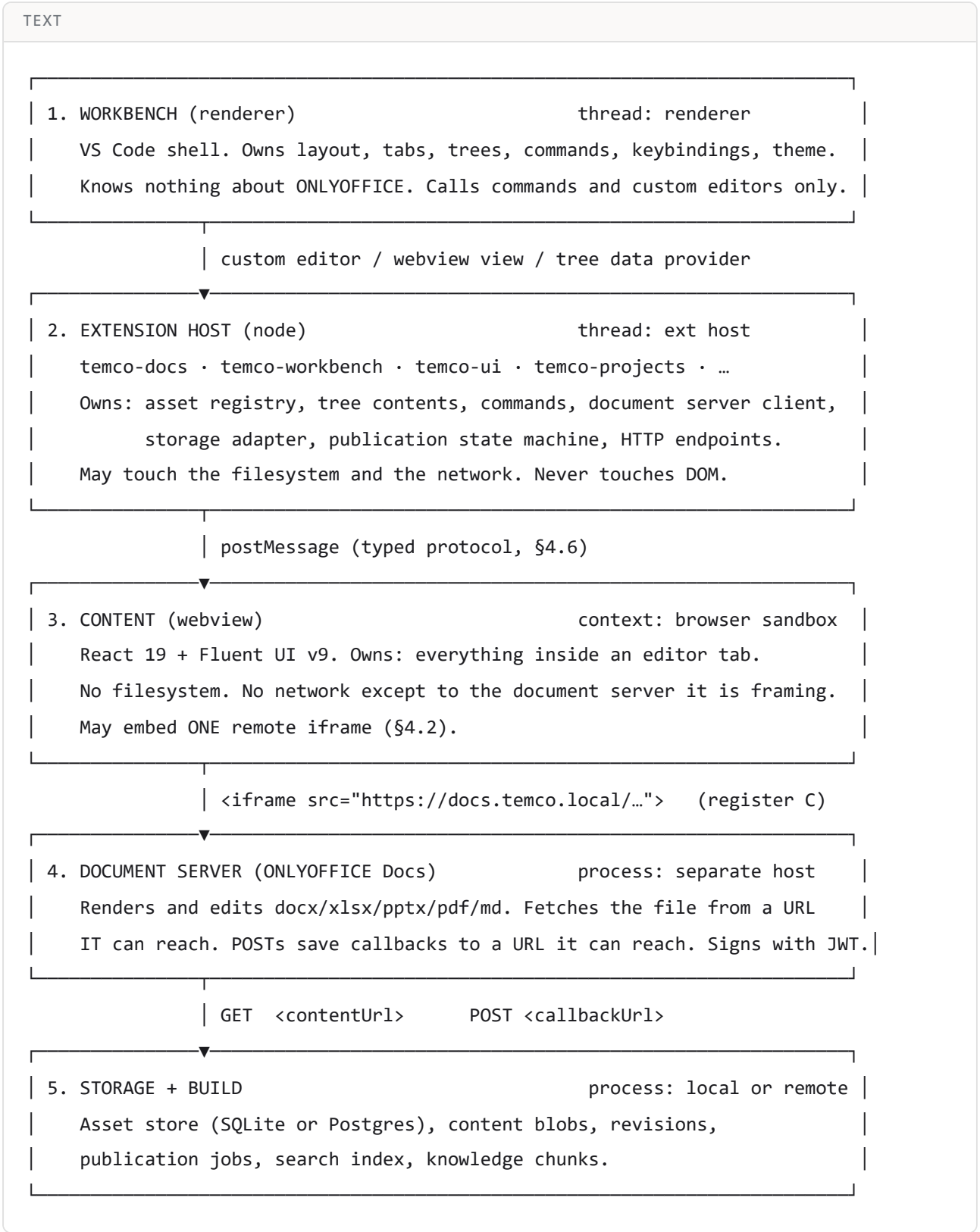
TEXT



3. Architecture

3.1 Planes and processes

TEXT



> **Plane 4 is optional.** In the embedded topology (§4.1.1 Topology B) it collapses into plane 3: the converter runs as `x2t.wasm` inside the webview, and planes 3 and 5 talk directly. Planes 1, 2, and 5 are identical either way, which is why the transport is pluggable rather than a fork in the design.

3.2 Hard truths this architecture imposes

These are not opinions. Each one has sunk an ONLYOFFICE integration before.

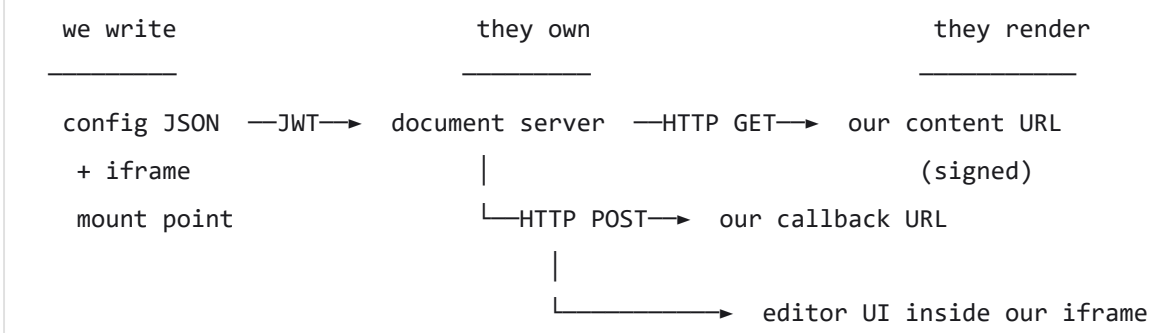
#	Truth	Consequence in code
T1	The document server, not the client, downloads the document	The content URL must be reachable **from the server**, not from the user's machine. `file://` and `blob:` cannot work.
T2	That fetch is unauthenticated by design	Content URLs must be short-lived signed URLs (HMAC + expiry, single use), never session cookies. §4.4
T3	Saves arrive as a POST to a URL the server can reach	The extension host must run a small HTTP listener with a stable address and a ticket per editing session. §4.5
T4	The editor's `key` identifies a *version*, and it is cached	Reusing a key after content changes serves stale bytes. The key must be derived from `(assetId, revision, optionsHash)`. §4.3
T5	Callback status 2 must be answered and the bytes downloaded within ~10 s	Download starts immediately; the store write may finish later, but the fetch may not. §4.5
T6	vsix cannot ship the document server	Deployment is a decision (§4.9), not an implementation detail. Develop against local Docker, ship against an intranet host.
T7	A cross-origin iframe owns the keyboard while focused	Every host-side shortcut must be routed through iframe focus state (§12.3) or it will silently not fire.

4. The editor engine: ONLYOFFICE Docs

4.1 What we are integrating

ONLYOFFICE Docs (the "Document Server") is a **web application** — a separate service, typically `onlyoffice/documentserver` in Docker, listening on 80/443. It renders and edits office files in the browser and exposes them for embedding through `DocsAPI.DocEditor`. It is not a JS library. There is no `npm i` that gives us a Word editor.

TEXT



4.1.1 Two ways to run it: server-hosted, or fully embedded

> **Correction (recorded on purpose).** An earlier revision of this document claimed the engine "cannot be embedded in a VSIX". That was wrong. It is a common belief and it is worth writing down why it is false, because it changes the architecture.

Facts, checked:

1. **The editor UI is static files.** `web-apps/` + `sdkjs/` are HTML/JS/CSS. They can be served from any origin, including a `vscode-webview://` resource root or a `file://` side-car folder.
2. **The only native piece is `x2t`**, the converter that turns `.docx` into the editor's internal model and back. Without it nothing opens.
3. **`x2t` has a WebAssembly build** maintained by CryptPad: cryptpad/onlyoffice-x2t-wasm. Conversion runs in the browser, in a Docker/Emscripten-produced `x2t.wasm`, with no native backend.
4. **A static build of the full editor exists**: cryptpad/onlyoffice-builds.
5. **CryptPad ships the complete ONLYOFFICE editor with no document server**: assets are static, conversion is WASM in the browser, collaboration is CryptPad's own CRDT layer. This is production software used by many organisations.

So the question is not *can it run without a server* — it can. The question is **which topology we can support and license.**

TEXT

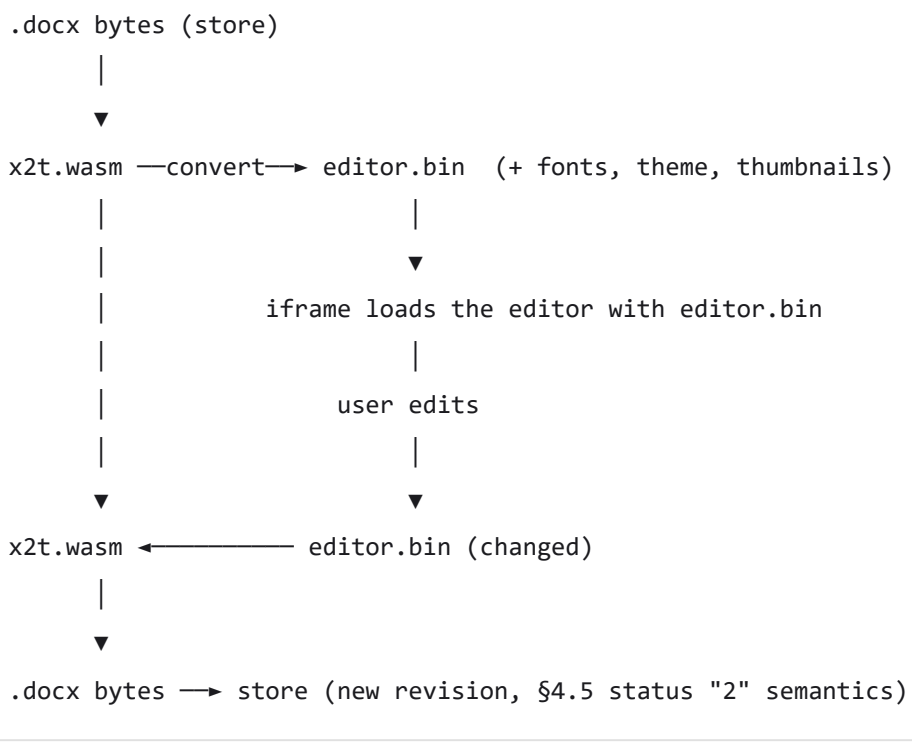
```
TOPOLOGY A — server-hosted (default)

┌──────────────────────────────────────────────────────────────────────────┐
│ VS Code — webview — <iframe src="{server}/web-apps/…">                   │
│                          │                                               │
│                          ▼                                               │
│             Document Server (nginx + node + x2t native + Redis/Postgres) │
│                 GET content   ·   POST callback   ·   co-editing websockets │
└──────────────────────────────────────────────────────────────────────────┘


TOPOLOGY B — fully embedded (no server anywhere)
┌─ VS Code (Electron) ─────────────────────────────────────────────────────┐
│  extension host                                                          │
│    └ extension resource dir                                              │
│        onlyoffice/web-apps/…      static editor UI                       │
│        onlyoffice/x2t.js + x2t.wasm   converter, in-process              │
│        onlyoffice/fonts/…         fonts the renderer expects             │
│                                                                          │
│  webview (register B — Fluent host)                                      │
│    └ <iframe src="vscode-webview://…/onlyoffice/web-apps/…">             │
│         editor UI runs locally · zero network                            │
│                                                                          │
│  asset store  (bytes in, bytes out)                                      │
└──────────────────────────────────────────────────────────────────────────┘
```

Embedded open/save pipeline:

TEXT



4.1.2 What actually decides it

Constraint	A — server-hosted	B — fully embedded
Licence	AGPLv3 service, run unmodified, separate process	**AGPLv3 code redistributed inside our product**
Download size	0 MB to the client	~200–400 MB (assets + fonts + wasm)
Cold start	3–9 s (server render)	2–6 s (wasm init ~1–2 s + conversion)
Co-editing	built in	**none** — needs our own CRDT, i.e. new engineering
PDF export	server-side, reliable	via `x2t.wasm`, limited/canvas-dependent
Update path	upgrade one server	every client re-downloads
Support	official, vendored	community fork; **we own the build** (Docker + Emscripten + fs shim + font pack)
Offline	no	**yes**
Data leaves the machine	no (self-hosted)	never

The licence is the real question, not the technology. ONLYOFFICE DocumentServer is AGPLv3. Two routes:

- **Internal use (no distribution outside the company):** embedding is fine in practice. AGPL obligations attach to *conveying* the program to third parties, and to modified versions offered over a network. An internal deployment does neither. Confirm with counsel, but this is the usual industry reading.
- **Distributed product (sold or given to customers):** either buy a commercial ONLYOFFICE licence before shipping the binaries, or ship Topology A where the AGPL program stays a separate service we run.

Both routes are viable. Topology A ships sooner and keeps co-editing; Topology B removes the server entirely and is the only option if policy forbids standing one up.

4.1.3 Therefore: the transport is pluggable

Everything above the transport is identical, so we do not choose permanently. One interface, two implementations, chosen by configuration.

TYPESCRIPT

```
interface OfficeTransport {
        readonly id: 'http' | 'embedded';
        readonly capabilities: {
                coEditing: boolean;        // true for http, false for embedded
                comments: boolean;
                exportPdf: 'server' | 'wasm' | 'none';
                offline: boolean;
                maxDocBytes: number;
        };
        /** Produces the config (and, for 'http', the signed JWT) the surface needs. */
        prepare(asset: AssetRecord, revision: number, mode: 'edit' | 'view'): Promise<O
        /** Resolves when the surface has reported a saved revision. */
        awaitSave(session: OfficeSession): Promise<Revision>;
        dispose(session: OfficeSession): Promise<void>;
}
```

Consequences, all of which keep the code honest:

- **TR-1** The host↔content protocol (§4.6) does not change. `init`, `saved`, `reload`, `error` mean the same thing in both.
- **TR-2** The `key` rule (§4.3) survives: it still identifies a version and must change when the revision changes. In embedded mode there is no server cache, but the editor still uses it to decide whether to reuse in-memory state.
- **TR-3** In embedded mode the "save callback" is a local function, not an HTTP POST. It still writes an immutable revision (§6.1 D1–D2) and it still must satisfy the 10-second rule's *intent*: convert and write off the UI thread, never block the canvas.

- **TR-4** Capability differences must be visible, not silent: with `coEditing: false` the header hides the presence avatars and the asset shows `single-user` in Properties.
- **TR-5** A test opens the same fixture through both transports and asserts the round-trip law (§14.2). If a feature only works in one, it is listed in §4.1.2 rather than discovered later.

4.1.4 Decision (recorded): both transports, embedded is the default

Chosen because a server is something the team would rather not run, and because the product may later be sold.

	Value	Consequence
Transports supported	`embedded` **and** `http`	One interface, two implementations, no fork in the design
Default	`embedded`	Works offline, no service to operate, no tickets or callbacks
`http`	opt-in per workspace	The escape hatch for co-editing and for the packaged-build licence story
Co-editing	**not available** in the default	Must be stated in the UI, not discovered (§4.1.3 TR-4)
Distribution	internal now, possibly commercial later	Forces the licence gate below

Because "may be sold later" is on the table, the embedded build needs a **build-time gate**, not a promise in a document.

4.1.5 Shipping the embedded build: the office pack and the licence gate

The 300 MB does not belong in a VSIX. It ships beside the app.

TEXT

```
┌─ what ships, and how big it is (measured on this machine) ──────────────┐
│                                                                         │
│  VS Code fork (Electron)                                                │
│   Electron runtime                        ~360 MB   already shipped today │
│   resources/app/out/…                     ~120 MB   production build    │
│   resources/office-pack/                  ~300 MB   ONLYOFFICE assets   │
│     web-apps/   sdkjs/   fonts/   x2t.wasm            → §4.1.6 to shrink  │
│     LICENSE.txt   NOTICE.txt              ← attribution, always present │
│                                                                         │
│  Extensions (temco-docs)                                                │
│   out/…                                   ~2 MB     extension code      │
│   locates the pack via officePackPath(); if absent, capability = false  │
└─────────────────────────────────────────────────────────────────────────┘
```

The gate runs in the build, so a mistake fails CI instead of becoming a legal problem:

TEXT

```
build.mjs
  │
  ├─ TEMCO_OFFICE_EMBED=1 ?  ──no──►  do not copy the pack.
  │                                    embedded transport reports
  │                                    { available: false }  → §4.10 banner
  ▼ yes
  ├─ TEMCO_OFFICE_LICENCE = ?
  │        │
  │        ├─ agpl-internal ──► allowed only when TEMCO_BUILD_CHANNEL=internal
  │        │                    CI fails if channel = public
  │        │
  │        ├─ commercial ─────► allowed in any channel;
  │        │                    NOTICE.txt records the licence reference
  │        │
  │        └─ unset ──────────► CI fails: "embedded pack requires a licence decision"
  ▼
  copy pack → write NOTICE.txt → expose in About › Third-party notices
```

Enforceable rules:

- **LG-1** A public-channel build containing `resources/office-pack/` without `TEMCO_OFFICE_LICENCE=commercial` **fails the build**. No warning, a failure.

- **LG-2** `LICENSE.txt` and `NOTICE.txt` from the ONLYOFFICE distribution are copied unmodified and are never excluded by the packager.
- **LG-3** The About dialog lists the pack version and licence mode; Diagnostics (§10.12) can export them.
- **LG-4** Missing pack is a **degraded state**, not a crash: Office assets open **read-only** with the §4.10 banner and a one-line reason, and offer Download. There is no silent fallback to the text editor — that is only ever an explicit `Open With → Text Editor` choice, because editing a `.docx` as bytes destroys it.
- **LG-5** The pack is version-pinned to the editor build (`x2t.wasm` and `web-apps` must come from the same ONLYOFFICE release); a mismatch is reported as `office-pack-mismatch` .

The internal/commercial switch is exactly the point the user raised: internal use does not trigger AGPL distribution obligations, shipping does — so the build system, not memory, decides which one is happening.

4.1.6 Size: where it comes from, and how to shrink it

Measured on the development machine, so these are facts rather than round numbers:

Part	Size	Note
Electron runtime	**360 MB**	already in the product today; not caused by ONLYOFFICE
VS Code product (`out/`)	**267 MB dev** / ~120 MB production	source maps are most of the dev figure
ONLYOFFICE office pack	**~300 MB estimate**	`web-apps` + `sdkjs` + fonts + `x2t.wasm`

Why the upstream package is so heavy: the Document Server image is **2–3 GB+** because it bundles editors for word/cell/slide/pdf, the whole `sdkjs` engine for each, dozens of locale packs, a large font collection (a major contributor — it is what makes documents look right), the `x2t` converter, and nginx/node/redis/postgres. Nothing in there is waste; it is just everything.

Shrink levers, largest first:

Lever	Est. saving	Cost
Ship **word only** (drop cell/slide/pdf editors)	~35–45 %	those formats open as download only
Prune locales to `en-US` + `zh-CN`	~10–20 %	untranslated UI for other languages
Prune fonts to a curated set (~20 families)	~30–50 %	font substitution → layout drift on documents using exotic fonts
Ship the pack as a compressed archive, extract on first run	download ↓50–60 %	install time; disk use unchanged
Download the pack on first use ("Office Pack" component)	installer **0 MB**	needs a download channel; first open needs network
Do several of the above together	**~60–120 MB plausible**	measure, do not assume

Order of preference: **word-only + two locales + curated fonts + first-run download.** It keeps the installer small and the runtime honest, and it is reversible — the pack is a versioned artifact, not code.

Licence is not MIT. ONLYOFFICE DocumentServer is AGPL-3.0 (verified from the repository `LICENSE`). The `@onlyoffice/document-editor-react` wrapper and some helper packages carry their own permissive licences, which is where the MIT impression comes from — but the editors, `sdkjs`, and `x2t` are AGPL. Nothing that does real OOXML layout is MIT; MIT-licensed packages in this space (`docx-preview`, `docx`) render or generate, they do not edit with fidelity.

If either size or licence turns out to be unacceptable, these are the engine alternatives — evaluated as engines, not as libraries:

Engine	Licence	Size	DOCX fidelity	Note
ONLYOFFICE, embedded	**AGPL-3.0**	~300 MB, reducible	excellent	we own the build
ONLYOFFICE, server	AGPL-3.0, separate service	0 MB on the client	excellent	needs a host; the only topology with co-editing
ZetaOffice (LibreOffice in WASM)	**MPL-2.0** — file-level copyleft, **no obligation on our app**; `zetajs` is MIT	large, same order	very good	commercial support from allotropia/CIB; the licence-safe swap for Topology B
Syncfusion Document Editor	commercial (community licence for small teams)	~10–20 MB	good on common documents	per-developer licence
TX Text Control	commercial	~50 MB	very good	per-developer licence
Apryse / Nutrient WebViewer	commercial	varies	good, conversion-centric	per-server licence
Tiptap / Milkdown	MIT	tiny	document model, not page layout	already rejected for Word fidelity (§4.8)

The decision rule: if AGPL blocks distribution, swap the engine for **ZetaOffice** and keep Topology B — the `OfficeTransport` interface absorbs it. If *size* is the problem, only a commercial component (Syncfusion or TX) actually solves it, and that is a budget conversation, not an engineering one. Keep this table current; re-read it before P5 when the product might ship.

4.1.7 The zero-AGPL routes, and what "mock the editor" can mean

AGPL-3.0 does not forbid *using* the software. It forbids **conveying** it without the corresponding source, and (for modified versions) offering it over a network without source. So there are four clean routes, and only two of them involve settling anything with ONLYOFFICE.

#	Route	AGPL exposure	Client size	DOCX fidelity	Cost
R1	**Separate service** (Topology A): the AGPL program runs as its own process; we only open an iframe to it	**None** — no conveyance, no derivative work	**0 MB**	excellent	a host to run
R2	**Commercial ONLYOFFICE licence** → embed Topology B legally	None (licensed)	~300 MB, reducible	excellent	licence fee
R3	**MPL engine** (ZetaOffice = LibreOffice in WASM) → embed	None on our code (file-level copyleft only)	large	very good	optional support contract
R4	**No office engine at all** — our own editor + MIT viewers	**None**	**~2 MB**	none for imported docs	engineering time

Route **R4** is the one worth taking seriously, because a VS Code fork with a Fluent UI is already a very good document authoring surface — that is what the Markdown host in §4.8 is.

4.1.7.1 The question that decides it

Not "which engine" — but **must existing `.docx` files be edited in place inside the app?**

TEXT

```
┌─ what the corpus needs ──────────────────────────────────────────────┐
│                                                                      │
│  A. AUTHOR new documents          → Fluent + Tiptap (MIT)            │
│     our own .md / .temco format       zero AGPL, ~1 MB, 50 ms open   │
│                                                                      │
│  B. VIEW an imported .docx        → docx-preview (MIT) or mammoth (BSD)  │
│     read-only, no fidelity promise     zero AGPL, ~300 KB            │
│                                                                      │
│  C. EDIT an imported .docx        → pick one:                        │
│     exact page layout must survive                                   │
│                                       1. ONLYOFFICE as a separate    │
│                                          service ................ R1 │
│                                       2. commercial licence ...... R2 │
│                                       3. ZetaOffice, MPL-2.0 ...... R3 │
│                                       4. convert-on-import, edit ours, │
│                                          export .docx (lossy) .... R4 │
└──────────────────────────────────────────────────────────────────────┘
```

Routing, as the editor host registry implements it (§4.11):

TEXT

```
open asset
   │
   ├─ .md / .temco ──────────► Fluent editor (MIT)                  [A]
   │
   └─ .docx
         │
         ├─ "View" ──────────► docx-preview in the webview (MIT)    [B]
         │
         └─ "Edit"
               │
               ├─ office service configured ──► ONLYOFFICE iframe   [R1]
               ├─ commercial licence ──────────► embedded pack      [R2]
               └─ neither ─────────────────────► "Convert to Temco Doc"
                                                  → editable here,
                                                  export back to .docx [R4]
```

R4 is honest about its trade: conversion is **lossy**. Tables survive, images survive, headings survive; exact pagination, floating shapes, and complex numbering generally do not. The original `.docx` is kept

untouched as a revision, and the conversion is an *opt-in* action with a visible warning — never a silent import.

4.1.7.2 What "mock the editor" can and cannot mean

Three different things hide behind that phrase. Only one of them ships.

Reading	Legal	Useful	Verdict
A clickable mock — a Fluent UI that looks and behaves like the editor, with no engine behind it	Yes, no third-party code at all	Yes — for stakeholder sign-off and for designing the flow before wiring an engine	**Do it** as a design artifact, never as the product
Mimic ONLYOFFICE's chrome — our own engine behind a ribbon like theirs	Functional layouts are not protected; their **icons, logos and fonts are**	The ribbon was never the hard part	Allowed, but buys nothing
A placeholder editor that quietly replaces the real one	Yes	No	**Forbidden** — someone will lose a document (see ZA-4)

If "mock" means the first one: yes, and it is a good idea — a Fluent mock is how we validate the flow in §10 without committing to an engine. It cannot be shipped as the editor, because a mock has no save path and therefore no revision.

4.1.7.3 Rules

- **ZA-1** No AGPL code enters the VSIX or the office pack unless R2 (commercial licence) is in place. Enforced by the §4.1.5 build gate.
- **ZA-2** Under R1 we do not fork or patch ONLYOFFICE. Configuration, JWT, and an iframe only. If we ever patch it, we publish the patch — the cheapest way to stay clean is to never touch their source.
- **ZA-3** If we *ship* the Document Server image to a customer, we are conveying it and must provide the corresponding source. Simplest compliance: the customer pulls `onlyoffice/documentserver` themselves; we ship only the URL and the secret.
- **ZA-4** No surface ever presents an editable-looking document without a working save path. A placeholder that accepts keystrokes and discards them is a data-loss bug, not a mock.
- **ZA-5** Import/convert never destroys the source: the original `.docx` stays as a revision, and the converted asset records `derived-from`.
- **ZA-6** No ONLYOFFICE name or logo in our UI beyond the attribution the licence requires. "Office document" is fine; "ONLYOFFICE" as our feature name is not.

4.1.7.4 Decision (recorded): R4, with R1 kept as the escape hatch

Corpus fact: **a few dozen existing `.docx`, edited occasionally** — not weekly round-trips. That does not justify a 300 MB dependency, a licence purchase, or a service to operate.

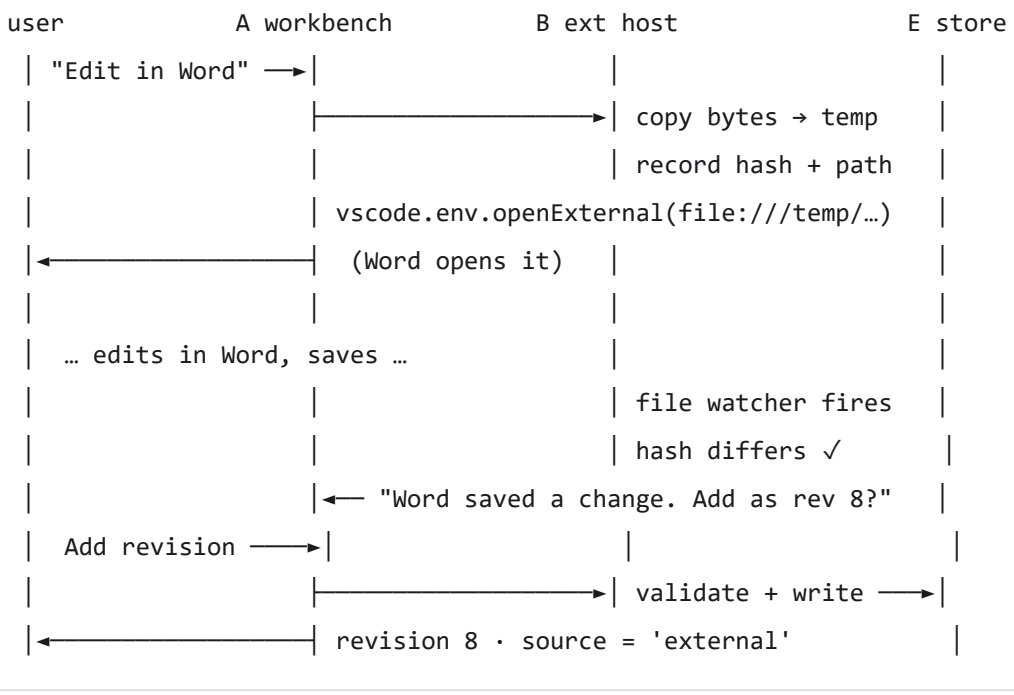
Item	Decision
Authoring	Fluent + Tiptap (MIT)
Viewing an imported `.docx`	`docx-preview` (MIT) in the webview, read-only
Editing an imported `.docx`	**Convert-on-import** (R4), lossy, opt-in, with a warning
Export	`.docx` generated from our model via the MIT `docx` package
Faithful editing of a complex original	**F12** — edit it in Word and re-import (below)
ONLYOFFICE	**not in the shipped product**; R1 becomes available the day a host exists
AGPL exposure in what we ship	**zero**

Two consequences worth stating plainly:

1. **The Markdown/Fluent editor is now the primary authoring surface**, so the caret-and-selection work already done there is the main line of investment, not a side quest. §10.5's acceptance criteria become P1 criteria.
2. **Fidelity is preserved by not competing with Word.** For the few documents that must look exactly right, the user edits in Word; we are the version store around it. That is cheaper and more honest than a lossy converter pretending otherwise.

F12 — Edit an original .docx in Word, bring it back as a revision

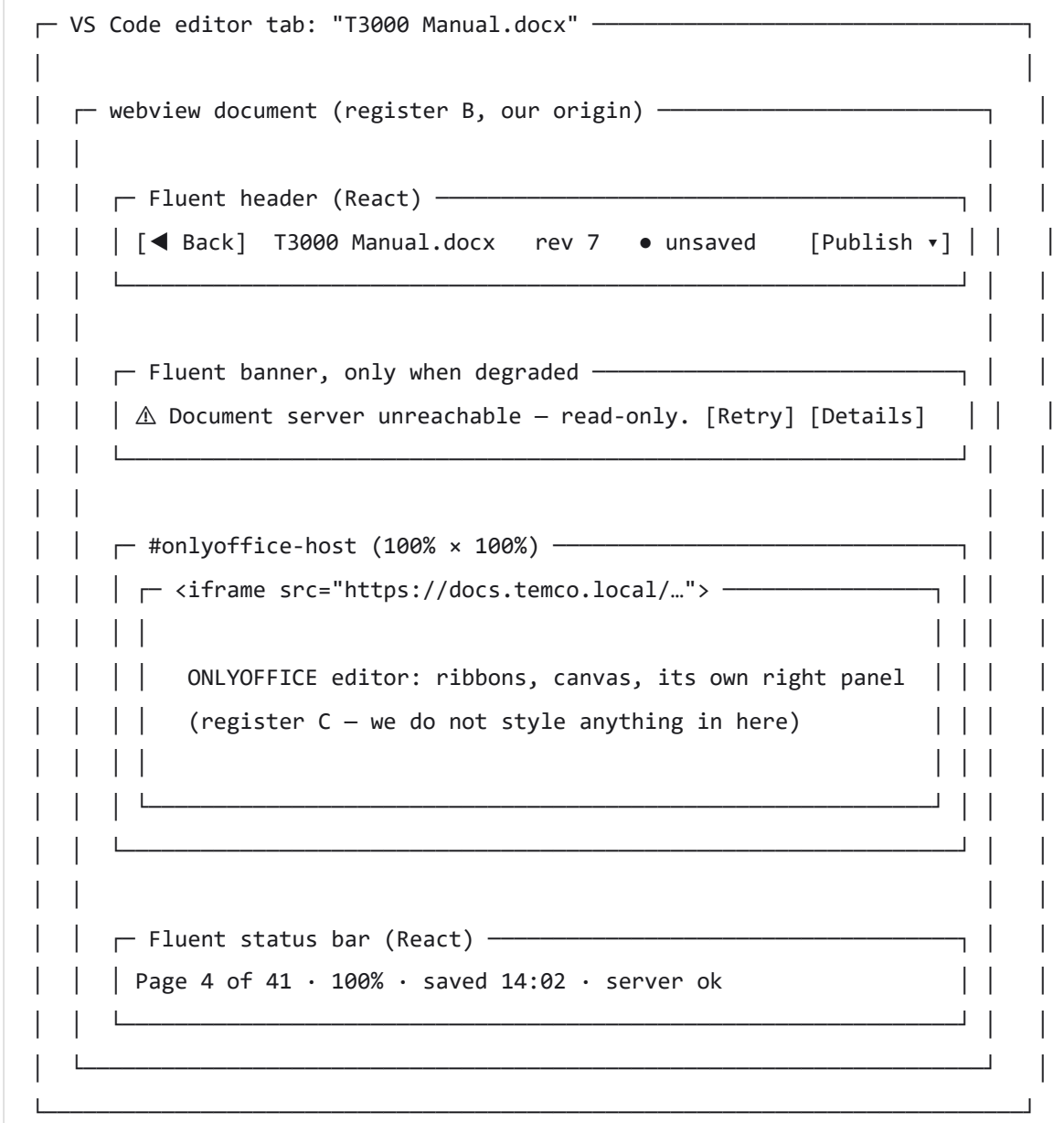
TEXT



Rules: the temp copy is the user's, never rewritten by us; a hash match produces no prompt and no revision (the round-trip law applies here too); the asset records `source: 'external'` so history shows it came from Word.

4.2 Composition inside a VS Code custom editor

TEXT



Only three things are ours: the header, the banner, the status bar. The document canvas is theirs. Do not attempt to inject CSS into the iframe — it is cross-origin; it will not work and it voids support.

4.3 The key — the only thing that causes corruption if you get it wrong

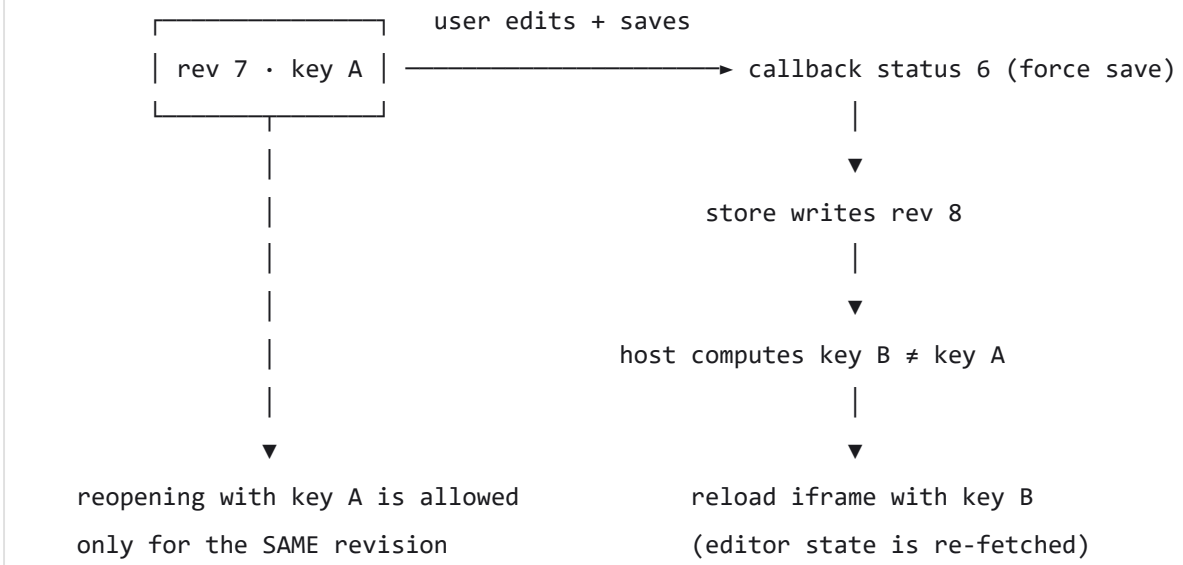
ONLYOFFICE caches a document by `key`. Same key means "same content", so it will reconnect the user to whatever it already has. Change the content without changing the key and users keep editing the old bytes and then save over the new ones.

TEXT

```
key = sha1( `${assetId}:${revision}:${optionsHash}` ).slice(0, 40)

assetId      AS-8812
revision     7                ← bumps on every accepted save callback
optionsHash  docx|word|edit   ← editing mode, language, permission set
```

TEXT



Invariants the code must enforce:

- **I1** — A key never outlives its revision. `revision` is part of the hash, so this is automatic.
- **I2** — A key is ≤ 128 chars and contains no `:`, `/`, or whitespace.
- **I3** — When two tabs open the same `(assetId, revision)`, they intentionally share a key: that is co-editing, and it is a feature. When the revision bumps, both are told to reload (§4.7 `onOutdatedVersion`).
- **I4** — Never derive the key from a timestamp or a random value. Doing so breaks co-editing and orphans the server's cache.

4.4 The content URL — reachable by the server, dead in 15 minutes *(http transport only)*

TEXT

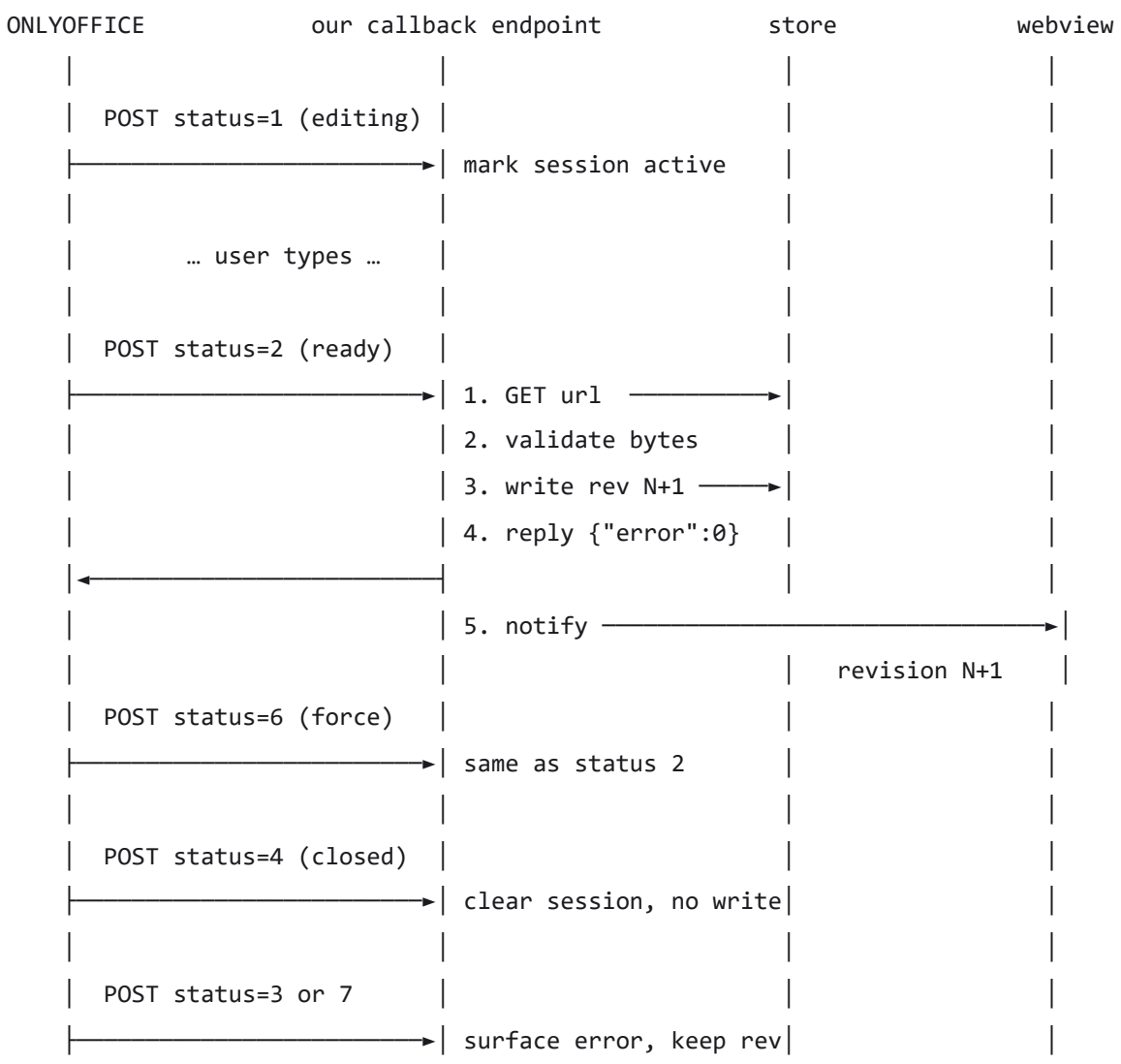
```
https://platform.temco.local/api/docs/content/AS-8812/7
    ?exp=1735660921
    &ticket=c7f3…            ← single-use, bound to asset+rev
    &sig=HMAC-SHA256(exp|ticket|asset|rev, CONTENT_SECRET)
```

Rules:

- The handler checks `exp`, checks the signature, consumes the ticket, then **streams the blob**. No session, no cookie, no `Authorization` header — the document server sends none of those.
- `Content-Type` must match the extension; the path must end in `.docx` / `.xlsx` / `.pptx` / `.pdf` / `.md` **or** `fileType` in the config must be set. Both is safer.
- Range requests must be supported (`Accept-Ranges: bytes`) — the server probes.
- The secret is per-deployment, stored in `SecretStorage`, never in settings JSON.

4.5 The save callback — status machine and the 10-second rule *(http transport only)*

TEXT



Status	Meaning	Our action
1	document being edited	record session start; no write
2	ready for saving, content changed	download, validate, write revision, reply `{"error":0}`
3	saving error	keep the old revision, raise a Fluent toast, reply `{"error":0}` so the server stops retrying
4	closed, no changes	clear session; **do not** write a revision
6	force save (autosave / version bump)	identical to 2
7	force-save error	as 3

The 10-second rule (T5). The HTTP GET of `url` must start before the callback response is written, and the response must not wait for the store write. Implement as: `await fetchBytes(url)` → reply → `await store.write(...)`. Reversing that order produces sporadic "document could not be saved" in the editor even though our store succeeded.

Validation before write (all four, or reject with status 3 semantics):

1. Non-empty body, and `Content-Length` under the asset-type limit.
2. Declared extension matches the opened format.
3. For Office types, the body starts with the correct zip magic `PK\x03\x04` (docx/xlsx/pptx are zips). A 200-with-HTML error page from a proxy is common; this catches it.
4. `assetId` and `revision` in the query match the session ticket; a replayed callback must be rejected.

4.6 Message protocol between extension host (2) and webview (3)

Typed, versioned, no free-form strings. Both directions are discriminated unions; unknown types are logged and dropped, never thrown.

TYPESCRIPT

```typescript
// host → webview
type HostToContent =
  | { t: 'init'; asset: AssetView; server: ServerState; config: OfficeConfig | null }
  | { t: 'server'; state: ServerState }
  | { t: 'saved'; revision: number; at: string }
  | { t: 'reload'; revision: number; reason: 'outdated' | 'forced' }
  | { t: 'error'; code: ErrorCode; message: string; detail?: string };

// webview → host
type ContentToHost =
  | { t: 'ready' }
  | { t: 'forceSave' }
  | { t: 'requestReload' }
  | { t: 'openExternal'; href: string }     // host validates the URL scheme
  | { t: 'log'; level: 'warn' | 'error'; message: string };
```

`OfficeConfig` is the already-signed ONLYOFFICE config. **The JWT is signed in the extension host** and never in the webview: the secret must not enter a browser context.

4.7 The config builder

TYPESCRIPT

```typescript
// extensions/temco-docs/src/office/config.ts
interface OfficeRequest {
        asset: AssetRecord;
        revision: number;
        mode: 'edit' | 'view';
        user: { id: string; name: string };
        endpoints: { contentUrl: string; callbackUrl: string };
        theme: 'light' | 'dark' | 'contrast';
}

export function buildOfficeConfig(r: OfficeRequest, secret: string): SignedOfficeConfig
        const fileType = r.asset.format;                        // 'docx' | 'xlsx' | …
        const config = {
                documentType: DOCUMENT_TYPE[fileType],              // 'word' | 'cell
                type: 'desktop',
                document: {
                        fileType,
                        key: documentKey(r.asset.id, r.revision, `${fileType}|${r.mode
                        title: r.asset.title,
                        url: r.endpoints.contentUrl,
                        permissions: {
                                edit: r.mode === 'edit',
                                download: true,
                                print: true,
                                comment: true,
                                review: true,
                                copy: true,
                        },
                },
                editorConfig: {
                        mode: r.mode,
                        lang: 'en',
                        callbackUrl: r.endpoints.callbackUrl,
                        user: r.user,
                        customization: {
                                uiTheme: r.theme === 'dark' ? 'theme-dark' : r.theme
                                compactHeader: true,
                                compactToolbar: true,
```

```
                                toolbarNoTabs: false,
                                hideRightMenu: true,
                                hideChat: true,
                                help: false,
                                about: false,
                                feedback: false,
                                autosave: true,
                                forcesave: true,
                            },
                            coEditing: { mode: 'fast', change: true },
                        },
                        width: '100%',
                        height: '100%',
                        events: { onDocumentReady: true, onError: true, onRequestClose: true, c
                    };
                    return { ...config, token: signJwt(config, secret) };
                }
```

`DOCUMENT_TYPE`: `docx|doc|odt|rtf|txt|md → 'word'`, `xlsx|xls|ods|csv → 'cell'`, `pptx|ppt|odp → 'slide'`, `pdf → 'pdf'`.

4.8 Format matrix — who edits what

Format	Editor host	Why
`.docx`, `.doc`, `.odt`, `.rtf`, `.txt`	**ONLYOFFICE word** (R1/R2/R3) or **convert-on-import** (R4, §4.1.7)	Fidelity, tracked changes, comments, page layout — or a lossy but AGPL-free path
`.xlsx`, `.xls`, `.ods`, `.csv`	**ONLYOFFICE cell**	Formulas, pivots, formatting
`.pptx`, `.ppt`, `.odp`	**ONLYOFFICE slide**	Master layouts
`.pdf`	**ONLYOFFICE pdf** (view)	Signatures later; never round-trip
`.md`, `.mdx`	**Both — per asset, remembered** (§4.8.1)	Default is the Fluent editor (50 ms open, frontmatter safe, offline). ONLYOFFICE when the asset is marked `office`.
`.json`, `.yaml`, `.xml`, `.bacnet`, `.ini`	**VS Code text editor**	Already perfect; leave it
`.drawio`	**Diagram editor** (phase 3)	Remote iframe, same host pattern
`.md` **preview**	Fluent Markdown editor, preview pane	Same surface
images	VS Code image preview	Already perfect

4.8.1 Per-asset host memory

Explicit choice beats a global preference: the same `.md` file may be prose for one author and tabular for another, and nobody should have to remember a setting.

TEXT

```
resolve(asset, explicitOverride?)
  │
  ├─ 1. explicitOverride          ← "Open With…" right now
  ├─ 2. asset.preferredHost       ← remembered from the last explicit choice
  ├─ 3. temco.docs.defaultHost    ← user setting ('auto' = skip to 4)
  └─ 4. registry priority         ← the table in §4.11
```

TYPESCRIPT

```typescript
// field on AssetRecord — see §6 for the full record
preferredHost?: 'office' | 'markdown' | 'text';
```

Rules:

- **H1** Choosing a host in *Open With…* writes `preferredHost` on the asset, so the next open is the same — this is the flow the user asked for ("when you choose … at the default md editor to change").
- **H2** `preferredHost` is metadata, not content. Changing it writes **no revision** and does not dirty the tab.
- **H3** `Open With… → Markdown editor (default)` clears the field rather than storing `'markdown'`.
- **H4** If the remembered host cannot serve the current format (e.g. `.xlsx` marked `markdown`), fall through to rule 4 and log one warning — never fail to open.
- **H5** The tree shows a small marker on assets whose host differs from the format default, so a `.md` that opens in ONLYOFFICE is not a mystery.

The cost is honest and accepted: **two engines to keep tested for `.md`**. §14.2 adds a matrix test that opens the same fixture through both hosts and asserts the round-trip law holds for each.

> **On Markdown and ONLYOFFICE.** ONLYOFFICE Docs 9.0 added native `.md` editing, and **the decision is to support both**, chosen per asset and remembered (§4.8.1). Fluent remains the *default* for four measurable reasons: frontmatter and non-standard fences do not round-trip through a document model; a 2 KB file pays a full editor session; there is no cursor/selection API we can drive for diagnostics; and offline authoring stops working. The escape hatch is real, not decorative: `Open With… → Office` on any markdown asset, then the choice sticks. If a future ONLYOFFICE release makes `.md` a first-class text round trip, the default can change — this row, not the architecture.

4.9 Deployment and licensing — decide before writing code

Option	Setup	Use for	Cost
Intranet Document Server (recommended)	One Docker host at whatever address the team can resolve (internal DNS, a VM IP, or a cloud URL — `docs.temco.local` is only a placeholder), TLS, JWT secret per environment	Production. All clients, one address, one co-editing session	Community edition is free with concurrency limits; white-label/branding needs a commercial licence
`localhost` Document Server	`docker run -i -t -d -p 8080:80 onlyoffice/documentserver` on each machine; the callback must then be reachable at `host.docker.internal`	Development, and single-user offline work	Docker required per machine; 10–20 s cold start; **no co-editing and no shared review**, because each user has an isolated server
ONLYOFFICE DocSpace / cloud	Their hosted API	If we never want to run servers	subscription
Embedded in the app (Topology B, §4.1.1)	Ship static `web-apps` + `x2t.wasm` as extension resources; no server at all	When policy forbids running a service, or offline is required	~200–400 MB download; **no co-editing**; AGPLv3 redistribution (fine internally, needs a commercial licence if the product is distributed); we own the build
Bundle Desktop Editors	A separate Electron app; cannot be embedded as a component	—	—

Two configuration values decide whether it works at all: `TEMCO_OFFICE_SERVER_URL` (a `https://` origin, no trailing slash) and `TEMCO_OFFICE_JWT_SECRET` (in `SecretStorage`). A missing or mismatched secret means every editor open fails with ONLYOFFICE's generic "The document security token is not correctly formed" — surface it in diagnostics (§11.9) with the words *"JWT secret mismatch between platform and document server*" so the cause is one search away.

4.10 Degraded states — never a blank frame

TEXT

server state	what the user sees	what still works
unknown	skeleton · "Connecting to document server…"	tree, other asset
reachable	the editor	everything
unreachable	MessageBar · "Read-only — server unreachable." [Retry] [Health check]	download, histor publish existing
auth-failed	"Token mismatch. An administrator must check the JWT secret." + Diagnostics link, no Retry loop	download, histor
too-old	"This needs ONLYOFFICE 9.0+ (server is X)."	download
server-error	"Document server returned 500." + correlation id	download
no-secret	"Document server is not configured."	all non-Office

Health probe: `GET {server}/healthcheck` with a **2 s** timeout, cached for 30 s, and **never** on the tree-loading path.

In embedded mode (Topology B, §4.1.1) there is no probe to run. It is replaced by a **capability check** — `x2t.wasm` loaded, fonts present, `SharedArrayBuffer` available (chromium needs cross-origin isolation for the threaded build) — and the same states apply: `unknown` while loading, `ready`, `unavailable` with the banner. Failure text names the missing piece rather than "server unreachable".

4.11 The editor host registry

One table decides everything. A new format is a new row, not a new code path.

TYPESCRIPT

```
interface EditorHost {
        readonly id: 'office' | 'markdown' | 'text' | 'diagram' | 'pdf';
        readonly priority: number;                      // higher wins a tie
        supports(asset: AssetRecord): boolean;
        readonly kind: 'remote-iframe' | 'webview' | 'vscode-native';
        open(asset: AssetRecord, ctx: OpenContext): Promise<void>;
}

// resolution order (§4.8.1): explicit Open With → asset.preferredHost → setting → highe
```

Registry contents at v1:

id	kind	matches	surface
`office`	remote-iframe	docx/xlsx/pptx/pdf/odt/ods/odp/csv	`temcoDocs.office` custom editor
`markdown`	webview	md/mdx	`temcoDocs.markdown` custom editor
`text`	vscode-native	everything else, and any explicit "Open as text"	VS Code text editor
`diagram`	remote-iframe	drawio	phase 3
`pdf`	remote-iframe, view-only	pdf	`temcoDocs.office`

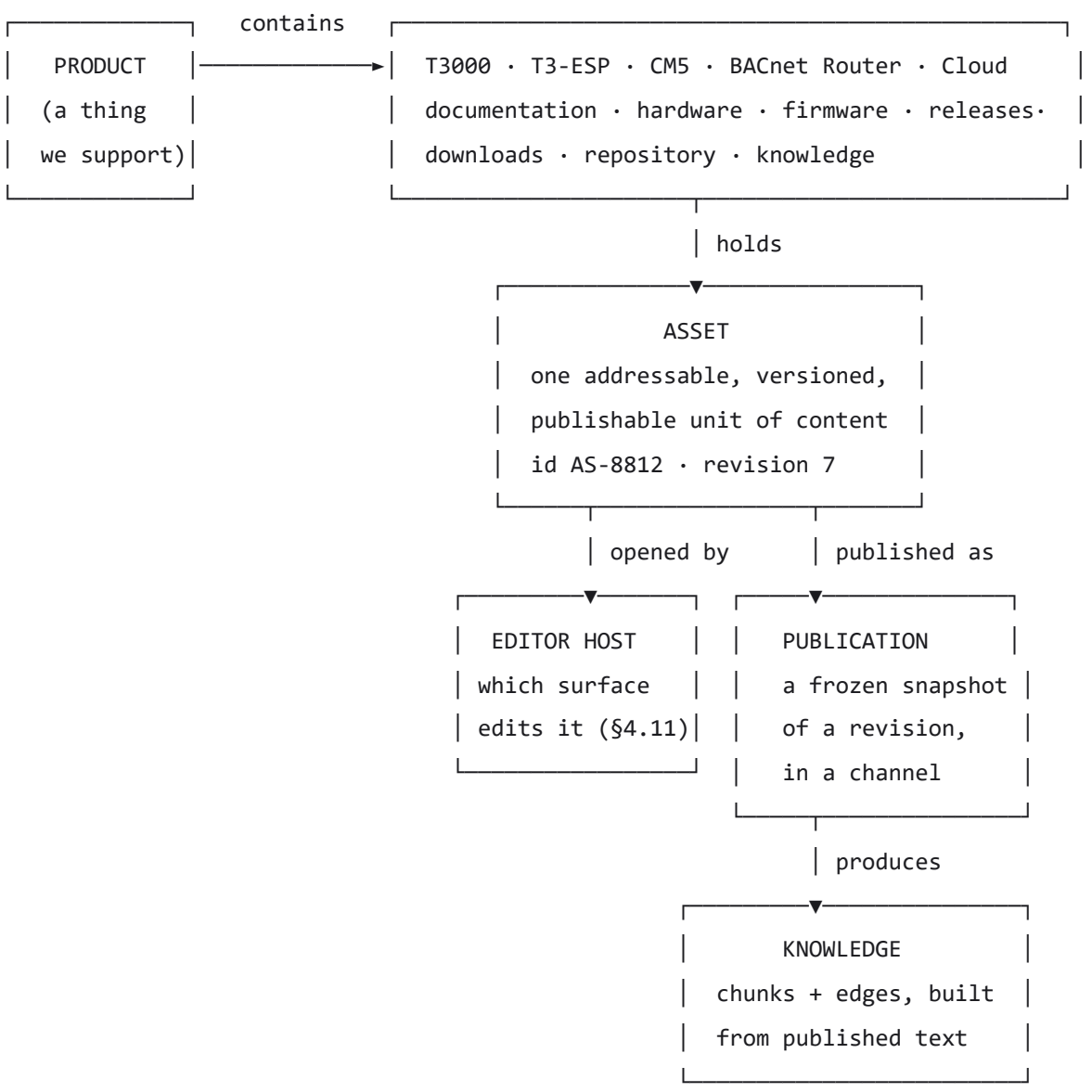
The "Open With…" entry on an asset's context menu lists these by display name, so a `.md` can be forced into Office and a `.docx` can always be forced into the text editor when a document is corrupt.

---

5. The five core concepts

Nothing else. If a requirement introduces a sixth noun, it is one of these five wearing a hat.

TEXT



Concept	One sentence	Never
Product	A thing we build and support for years	A folder
Asset	One addressable, versioned, publishable unit	A path on disk
Editor Host	The surface that opens an asset	A file extension check scattered in UI code
Publication	A frozen revision shipped to a channel	"Publish" that mutates in place
Knowledge	Derived index over published content	A hand-maintained second copy of the docs

5.1 The tree is the product, rendered by VS Code

```
TEXT
```

```
📦 T3000                          ← Product (register A: VS Code TreeView)
  📚 Documentation                ← Area
     Installation                 ← Document asset
     Commissioning                ← Document asset
     Troubleshooting/
        No comms with router      ← Document asset
  🔌 Hardware
     Inputs (12)                  ← Asset collection, opens a Fluent table
     Outputs (8)
     Variables (240)
  💾 Firmware
     13.0 · Latest
     12.9 · LTS
  📜 Releases
     13.0 · 2026-09
  ⬇ Downloads
```

Deterministic derivation, never hand-curated: `Product → areas(assetKind) → assets(area, parent)`. The provider is a `TreeDataProvider` in the extension host; it renders in register A and must never contain Fluent.

---

6. Domain model

TYPESCRIPT

```typescript
type AssetId = string;      // 'AS-8812'
type Revision = number;     // 1..n, monotonic, never reused

type ProductId = string;    // 'T3000'

type AssetKind =
        | 'document'      // docx/md/odt — the documentation body
        | 'spreadsheet'   // xlsx/csv
        | 'presentation'  // pptx
        | 'diagram'       // drawio/svg
        | 'pdf'
        | 'firmware'      // binary + version metadata
        | 'repository'    // link to a git remote
        | 'device'        // a physical controller model
        | 'input' | 'output' | 'variable' | 'program' | 'schedule' | 'graphic'
        | 'template'      // a starting point for a new asset
        | 'release'       // a named bundle of other assets
        | 'download';     // a file offered to customers

type Format =
        | 'docx' | 'doc' | 'odt' | 'rtf' | 'txt' | 'md' | 'mdx'
        | 'xlsx' | 'xls' | 'ods' | 'csv'
        | 'pptx' | 'ppt' | 'odp'
        | 'pdf' | 'drawio' | 'svg' | 'png' | 'jpg'
        | 'json' | 'yaml' | 'xml' | 'bin' | 'zip';

interface AssetRecord {
        id: AssetId;
        productId: ProductId;
        kind: AssetKind;
        format: Format;
        title: string;
        slug: string;                    // stable within (productId, kind, parentId)
        parentId: AssetId | null;        // tree shape for documentation
        area: string;                    // 'guides' | 'tutorials' | 'api' | … (§6.2)
        revision: Revision;              // current
        byteSize: number;
        updatedAt: string;
```

```
        updatedBy: string;
        /** Set only when the user picked a non-default host through Open With… (§4.8.1
        preferredHost?: 'office' | 'markdown' | 'text';
        /** Set when a publishable revision exists; null when never published. */
        publishedRevision: Revision | null;
}

interface RevisionRecord {
        assetId: AssetId;
        revision: Revision;
        bytes: Uint8Array;
        hash: string;                    // sha256 of bytes
        createdAt: string;
        createdBy: string;
        source: 'editor' | 'import' | 'restore' | 'template';
        message: string;                 // "Saved from Word editor", "Restored rev 5"
}
```

6.1 Invariants the store enforces

- **D1** — `revision` increases by exactly 1 per accepted save. Concurrent saves that would produce the same number fail with `ConflictError`.
- **D2** — Bytes are immutable once written. "Editing" a revision means appending a new one.
- **D3** — `publishedRevision` is always ≤ `revision`.
- **D4** — Deleting an asset deletes its revisions and removes it from every publication. It never deletes a published revision still referenced by a live publication.
- **D5** — `slug` is unique within `(productId, kind, parentId)` and, once a publication references it, changing it requires a redirect entry.

6.2 Areas — the documentation taxonomy

Mirrors MSDN, because the audience already knows it:

area	Holds	Example
`guides`	Task-oriented procedures	Commissioning a T3000
`tutorials`	Learning sequences	Your first BACnet schedule
`architecture`	Why the system is shaped this way	Polling model
`api`	Reference, one page per symbol	`SetOutput()`
`release-notes`	One page per version	13.0
`troubleshooting`	Symptom → cause → fix	No comms with router
`standards`	Internal conventions	Naming
`references`	Tables, pinouts, registers	Modbus register map

7. Fluent UI v9 inside VS Code

7.1 Pins and placement

Package	Version	Where
`@fluentui/react-components`	`^9.x` (pin exact, record it in `temco-ui/package.json`)	**`extensions/temco-ui`** — the only package that imports Fluent
`@fluentui/react-icons`	matching	`temco-ui`
`react` / `react-dom`	19.x (already in `temco-docs`)	`temco-docs`, `temco-ui`

`temco-ui` is currently an empty stub (no dependencies, no views). It becomes the design system package:

TEXT

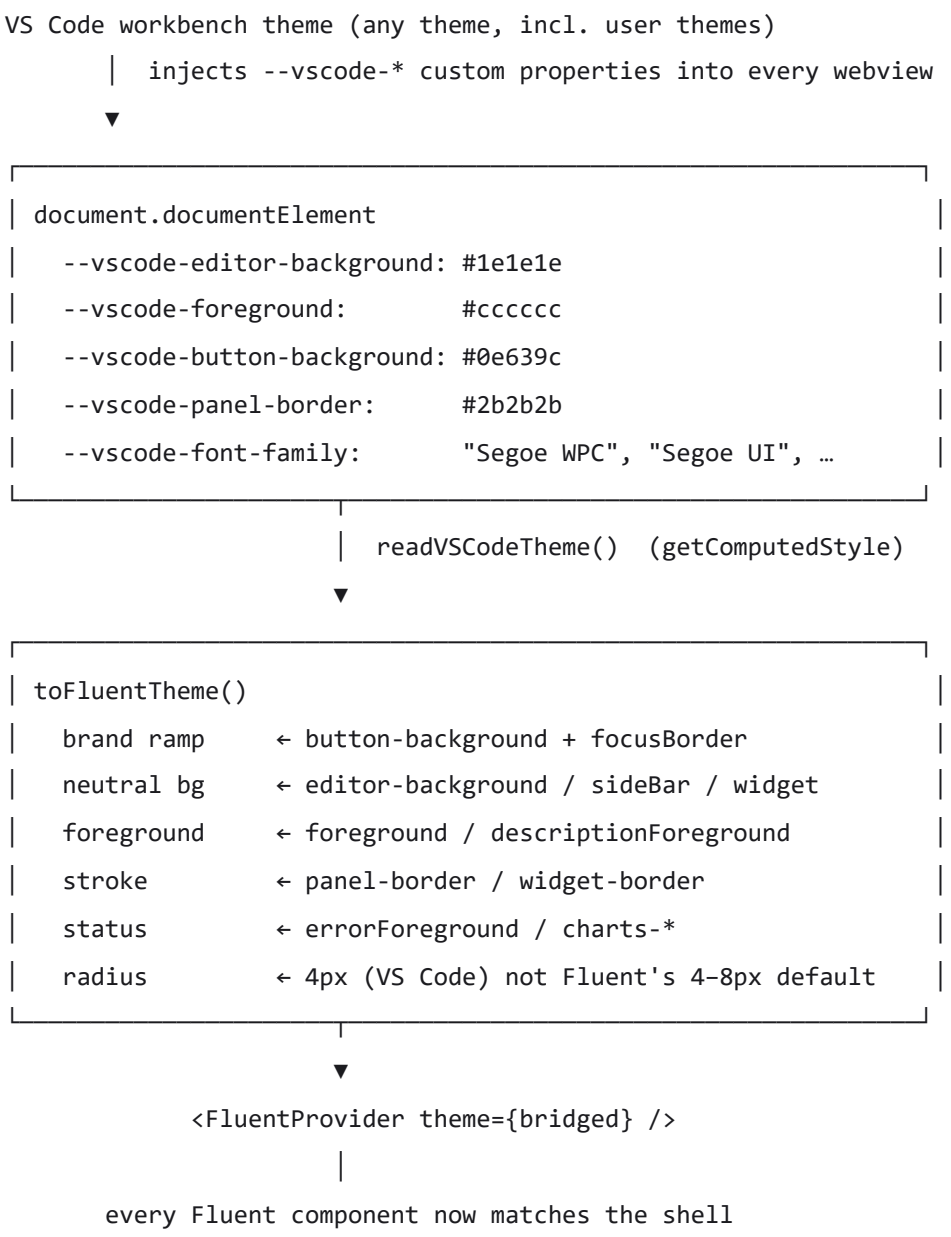
```
extensions/temco-ui/
  src/
    index.ts                 ← the ONLY public entry point
    provider/TemcoProvider.tsx    ← FluentProvider + theme bridge + density
    theme/readVSCodeTheme.ts      ← §7.2
    theme/toFluentTheme.ts        ← §7.2
    density/compact.ts            ← §7.4
    components/                   ← wrappers, not replacements
    patterns/                     ← EmptyState, ErrorState, StatusBar, DataTable
```

Import rule (lint-enforced): `@fluentui/*` may appear only in `extensions/temco-ui/src/**`. Every other extension imports `temco-ui`. This keeps one Fluent version in the bundle and one place to swap it.

7.2 The theme bridge — how Fluent stops looking foreign

VS Code publishes its colours as CSS custom properties (`--vscode-*`) on `:root` in every webview. Fluent publishes its own tokens in a JS theme object. The bridge reads the former and writes the latter.

TEXT


VS Code workbench theme (any theme, incl. user themes)
injects --vscode-* custom properties into every webview
document.documentElement
--vscode-editor-background: #1e1e1e
--vscode-foreground: #cccccc
--vscode-button-background: #0e639c
--vscode-panel-border: #2b2b2b
--vscode-font-family: "Segoe WPC", "Segoe UI", …
readVSCodeTheme() (getComputedStyle)
toFluentTheme()
brand ramp ← button-background + focusBorder
neutral bg ← editor-background / sideBar / widget
foreground ← foreground / descriptionForeground
stroke ← panel-border / widget-border
status ← errorForeground / charts-*
radius ← 4px (VS Code) not Fluent's 4–8px default
<FluentProvider theme={bridged} />
every Fluent component now matches the shell

TYPESCRIPT

```typescript
// extensions/temco-ui/src/theme/toFluentTheme.ts  (shape, not final numbers)
import { createLightTheme, createDarkTheme, type BrandVariants, type Theme } from '@flue

function read(name: string, fallback: string): string {
        const v = getComputedStyle(document.documentElement).getPropertyValue(name).tri
        return v === '' ? fallback : v;
}

export function brandRamp(): BrandVariants {
        const base = read('--vscode-button-background', '#0e639c');
        // ramp[10..160]; index 80 is the token Fluent uses for the accent
        return { 10: shade(base, -0.55), /* … */ 80: base, /* … */ 160: tint(base, 0.6)
}

export function toFluentTheme(kind: 'light' | 'dark' | 'hc'): Theme {
        const brand = brandRamp();
        const theme = kind === 'dark' || kind === 'hc' ? createDarkTheme(brand) : creat
        return {
                ...theme,
                fontFamilyBase: read('--vscode-font-family', 'Segoe UI'),
                fontSizeBase300: read('--vscode-font-size', '13px'),
                borderRadiusMedium: '4px',
                colorNeutralBackground1: read('--vscode-editor-background', theme.color
                colorNeutralBackground2: read('--vscode-sideBar-background', theme.colo
                colorNeutralForeground1: read('--vscode-foreground', theme.colorNeutral
                colorNeutralForeground3: read('--vscode-descriptionForeground', theme.c
                colorNeutralStroke1: read('--vscode-panel-border', theme.colorNeutralSt
                colorPaletteRedForeground1: read('--vscode-errorForeground', theme.colo
        };
}
```

Re-bridge when the theme changes. VS Code sets `document.body.dataset.vscodeThemeKind` (`vscode-light` | `vscode-dark` | `vscode-high-contrast` | `vscode-high-contrast-light`) and mutates `--vscode-*`. One `MutationObserver` on `body[data-vscode-theme-kind]` plus a one-frame settle is enough; debounce to 50 ms because a theme switch rewrites hundreds of variables.

Do not use `createLightTheme` / `createDarkTheme` alone. Unbridged Fluent is immediately recognisable as "a web page in a panel": different greys, different corner radius, different accent.

7.3 What we never do with Fluent

Forbidden	Why	Instead
`FluentProvider` wrapping shell DOM	Shell is not React	Provider is the root of each webview's React tree only
Fluent `Tree` for the side bar	Loses native accessibility, drag/drop, theming	VS Code `TreeView`
Fluent `Menu` for the tab context menu	That menu is owned by the shell	`menus` contribution
Fluent `Dialog` for a modal that blocks the whole window	Two modal systems, focus traps fight	VS Code command with `showQuickPick` / `showInputBox`, or a Fluent dialog **inside our webview only**
Fluent `Persona` with a photo	No avatar service; broken images	Initials chip
Fluent's default `fontFamilyBase`	Segoe UI ≠ VS Code's font stack	Bridged from `--vscode-font-family`

7.4 Density

VS Code rows are 22 px. Fluent defaults to 32–40 px. In `TemcoProvider`, apply a compact density for editor chrome and normal density for forms:

```typescript
// toolbar / status bar / property lists
<FluentProvider theme={theme} style={{ fontSize: '12px' }}>
        {/* size="small" on every input, Button, Dropdown, Tag, MenuButton */}
</FluentProvider>
```

Rules: toolbar controls `size="small"`; form controls `size="medium"`; all rows **22 px**; all icons **16 px**; no Fluent shadow on in-content surfaces (VS Code has no elevation inside editors except menus and tooltips).

7.5 CSP and Griffel

Fluent v9 uses Griffel, which injects `<style>` at runtime. The webview CSP must allow it, and the document server needs a frame slot:

HTML

```
<meta http-equiv="Content-Security-Policy" content="
  default-src 'none';
  img-src ${webview.cspSource} data: blob:;
  font-src ${webview.cspSource};
  style-src ${webview.cspSource} 'unsafe-inline';
  script-src ${webview.cspSource} 'unsafe-inline' ${officeOrigin};
  frame-src ${officeOrigin};
  connect-src ${webview.cspSource} ${officeOrigin};
  worker-src ${webview.cspSource} blob:;">
```

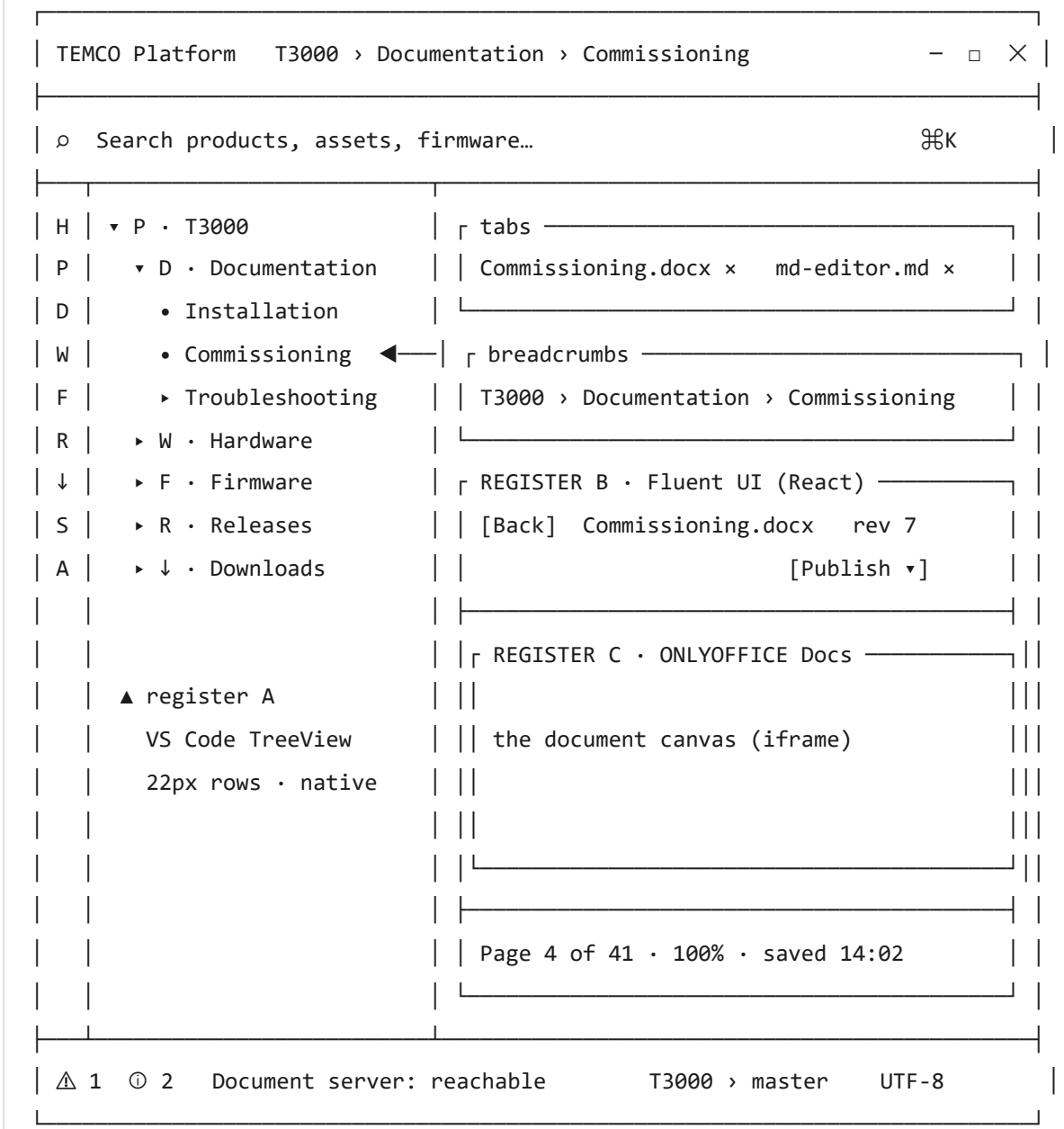
`${officeOrigin}` is the one configured document server origin — never `https:`. A `.docx` from a customer must not be able to reach a host we did not name.

---

8. The workbench

8.1 Full layout

TEXT



Activity bar legend: `H` Home · `P` Products · `D` Documentation · `W` Hardware · `F` Firmware · `R` Releases · `↓` Downloads · `S` Search · `A` Administration. Single-cell glyphs on purpose — emoji are double-width in some renderers and every border in this figure would drift.

The ONLYOFFICE status item is a **status bar item contributed by `temco-docs`** (§9.4) — register A, so it uses codicons and theme classes, not Fluent.

8.2 Activity bar — the whole platform, nine items

Order	Icon	id	Opens	Register
1	`home`	`temcoWorkbench.home`	Home webview view (recent, drafts, health)	B
2	`package`	`temcoWorkbench.products`	Product tree (the main explorer)	A tree
3	`book`	`temcoWorkbench.docs`	Documentation tree, filtered	A tree
4	`circuit-board`	`temcoWorkbench.hardware`	Hardware objects tree	A tree
5	`server`	`temcoWorkbench.firmware`	Firmware versions	A tree
6	`tag`	`temcoWorkbench.releases`	Releases	A tree
7	`cloud-download`	`temcoWorkbench.downloads`	Download catalogue	A tree
8	`search`	`workbench.view.search`	VS Code search, extended with an asset filter	A
9	`settings-gear`	`temcoWorkbench.admin`	Publication queue, diagnostics, health	B

Below the separator: AI (`sparkle`) from `temco-ai` , then Settings (built-in). **No custom activity bar rendering** — these are `viewsContainers.activitybar` contributions, so drag/drop, hide/show, and keyboard navigation come free.

8.3 Panel (bottom)

The existing VS Code panel hosts three contributed views: **Problems** (markdown lint), **Output → Temco Docs** (document server calls, one line per open/save), and **Publication log**. Nothing custom is drawn there.

---

9. From routes to VS Code constructs

Revision 1 had URLs. In a desktop host there are no URLs — there are **view ids, custom editor ids, commands, and a URI scheme**. This table is the contract; code must not invent a second way to reach a surface.

Concept (v1 route)	VS Code construct	id
`/` Home	webview view	`temcoWorkbench.home`
`/products`	tree view	`temcoWorkbench.products`
`/products/:id`	webview view, set by selection	`temcoWorkbench.product`
`/products/:id/docs`	tree view	`temcoWorkbench.docs`
`/doc/:assetId` (docx…)	**custom editor**	`temcoDocs.office`
`/doc/:assetId` (md)	**custom editor**	`temcoDocs.markdown`
asset properties	webview view in the *secondary* side bar	`temcoWorkbench.properties`
`/releases`	tree view	`temcoWorkbench.releases`
`/downloads`	tree view	`temcoWorkbench.downloads`
`/search`	built-in search + filter	`workbench.view.search`
`/review`	webview view	`temcoWorkbench.review`
`/publish`	webview view + command	`temcoWorkbench.publish`
`/admin`	webview view	`temcoWorkbench.admin`
`/ai`	webview view (`temco-ai`)	`temcoAi.chat`

9.1 The asset URI

Every asset needs an addressable identity so tabs restore, links work, and `Open With…` can be forced.

```text
temco-asset:/AS-8812?rev=7&host=office

scheme   temco-asset          registered by temco-docs; read-only, never on disk
path     /AS-8812             assetId
rev      revision (optional)  absent = latest
host     editor host id       absent = resolve via registry (§4.11)
```

Custom editors register for `viewType: 'temcoDocs.office'` and resolve through `CustomTextEditorProvider`-style `CustomEditorProvider` with `supportsMultipleEditorsPerDocument: false` (one editor column, principle §1.1).

9.2 Commands (25 for phase 1)

Command id	Title	Where
`temco.asset.open`	Open	tree item, `Enter`
`temco.asset.openWith`	Open With…	tree item context, `⌘⇧O`
`temco.asset.new`	New Asset…	tree view title
`temco.asset.newFromTemplate`	New From Template…	tree view title
`temco.asset.rename`	Rename	context, `F2`
`temco.asset.delete`	Delete…	context
`temco.asset.history`	Show History	context
`temco.asset.restoreRevision`	Restore Revision…	history
`temco.asset.download`	Download Copy…	context
`temco.asset.revealInTree`	Reveal in Tree	editor title
`temco.docs.forceSave`	Save Now	editor title, `⌘S`
`temco.docs.reload`	Reload From Server	editor title
`temco.docs.healthCheck`	Document Server Health Check	command palette
`temco.docs.diagnostics`	Show Document Server Diagnostics	command palette
`temco.publish.start`	Publish…	asset context, editor title
`temco.publish.queue`	Open Publication Queue	admin view, status bar
`temco.publish.cancel`	Cancel Publication…	queue item
`temco.product.add`	Add Product…	products view title
`temco.search.inAsset`	Find in Asset…	editor, `⌘F`
`temco.import.file`	Import Files…	tree view title
`temco.template.saveAs`	Save As Template…	asset context
`temco.docs.refresh`	Refresh	tree views
`temco.docs.signOutOffice`	Forget Document Server Session	diagnostics
`temco.ai.explainAsset`	Explain This Asset	editor title (`temco-ai`)

Command id	Title	Where
`temco.ai.findRelated`	Find Related Assets	context (`temco-ai`)

9.3 Settings

Key	Type	Default	Meaning
`temco.docs.serverUrl`	string	`""`	Document server origin, e.g. `https://docs.temco.local` (http transport only)
`temco.docs.transport`	enum	`embedded`	`embedded` | `http` | `auto` — `auto` uses http when `serverUrl` is set **and** reachable, else embedded
`temco.docs.officePackPath`	string	`""`	Override for the embedded office pack; empty = beside the app (§4.1.5)
`temco.docs.editorTheme`	enum	`auto`	`auto` | `light` | `dark` | `contrast` — forwarded to ONLYOFFICE `customization.uiTheme`
`temco.docs.autosaveSeconds`	number	`30`	0 disables; only affects ONLYOFFICE `autosave` hints
`temco.docs.markdown.engine`	enum	`fluent`	`fluent` | `office` — the §4.8 escape hatch
`temco.docs.defaultHost`	enum	`auto`	`auto` | `office` | `markdown` | `text`
`temco.docs.maxRevisionBytes`	number	`52428800`	Reject larger saves with a clear message

The **JWT secret is not a setting.** It lives in `SecretStorage` and is set by an administrator through `temco.docs.diagnostics`.

9.4 Status bar items

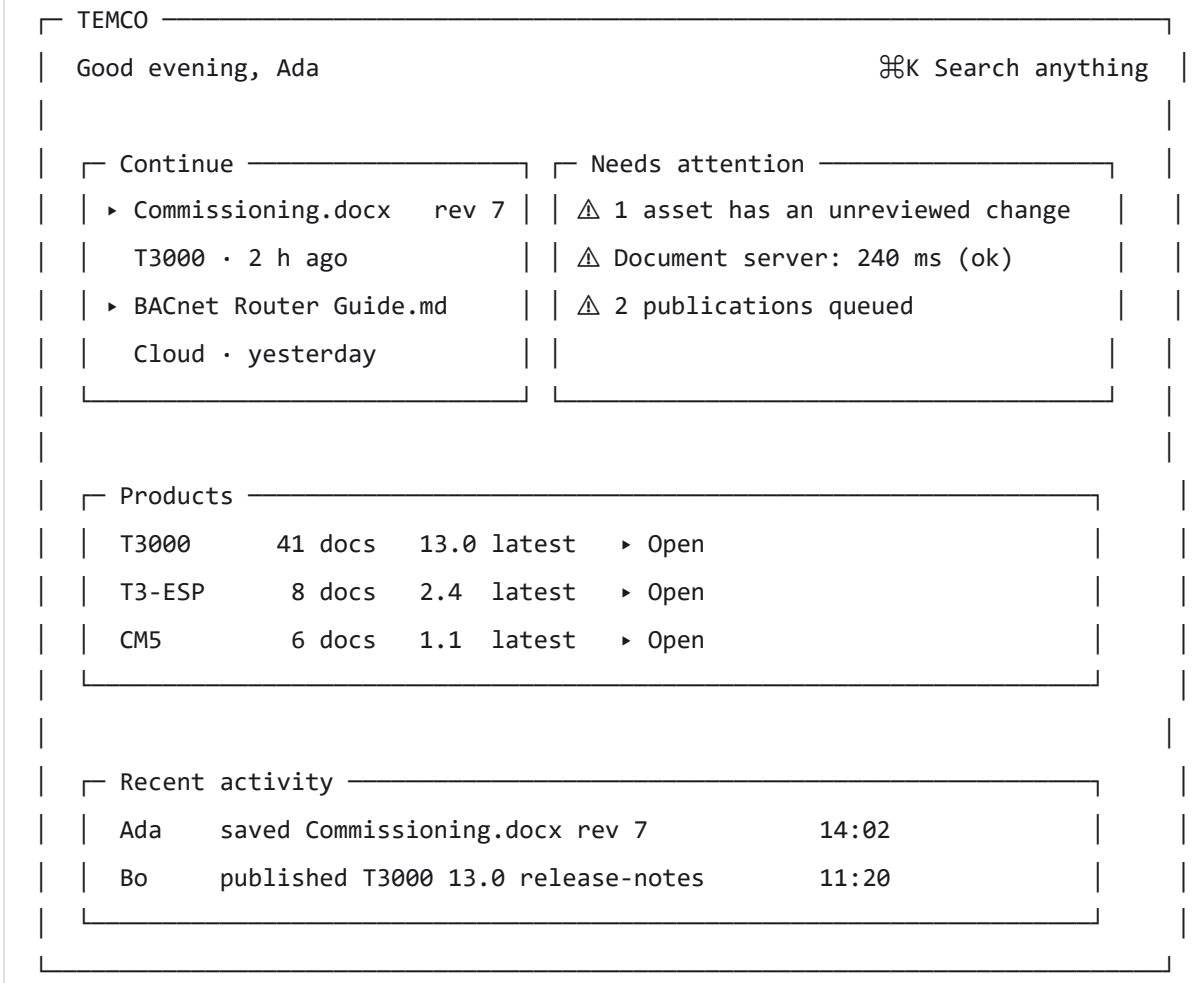
Item	Text	Click
Server	`$(plug) Document server` / `$(debug-disconnect) Document server` + tooltip with latency	`temco.docs.healthCheck`
Save	`$(cloud-upload) Saving…` → `$(check) Saved 14:02` — only while an Office editor is open	`temco.docs.forceSave`
Queue	`$(tag) 2 queued` — only when publications are pending	`temco.publish.queue`

10. Screens

Twelve surfaces. Each is either a **tree** (register A) or a **webview** (register B). None is drawn with a third stack.

10.1 Home — `temcoWorkbench.home` (webview)

TEXT



Fluent: `Text` (display), `Card`, `Badge`, `Button`, `MessageBar`, `Divider`, `Tag`. States: loading → 6 skeleton rows; empty (no products) → `EmptyState` with *Add Product*; server down → `MessageBar intent="warning"` above Products only.

Acceptance: first meaningful paint from a cold extension host under 600 ms; no document server call on this screen.

10.2 Product tree — `temcoWorkbench.products` (register A)

VS Code `TreeDataProvider`. Rows are 22 px, icons are codicons, selection drives the secondary side bar. Every row has a stable `id` so expansion survives refresh.

TEXT

```
▾ T3000                      product · 41 docs · firmware 13.0
    ▾ Documentation          area · guides (18 assets)
        Installation         docx · rev 7
        Commissioning        docx · rev 7
      ▸ Troubleshooting      area · 4 assets
    ▸ Hardware               12 inputs · 8 outputs · 240 variables
    ▸ Firmware               13.0 latest · 12.9 LTS
    ▸ Releases               3
    ▸ Downloads              14
▸ T3-ESP                     product · 8 docs
▸ CM5                        product · 6 docs
```

Selection drives the secondary side bar (§10.6); `Enter` opens the asset through F1.

Acceptance: expanding a product with 300 assets stays responsive (< 100 ms per expansion); no network call per row (batch-load children).

10.3 Product overview — `temcoWorkbench.product` (webview, secondary)

TEXT

```
┌─ T3000 ──────────────────────────────── [Open Docs] [New ▾] [Publish ▾] ─┐
│ 41 documents · 12 inputs · 8 outputs · firmware 13.0 · last published 09-14 │
├──────────────────────────────────────────────────────────────────────────┤
│ Tabs: Overview │ Documentation │ Hardware │ Firmware │ Releases │ Downloads  │
├──────────────────────────────────────────────────────────────────────────┤
│ Documentation by area          │ Recently changed                        │
│  guides            18          │  Commissioning.docx        rev 7  2 h   │
│  tutorials          6          │  Troubleshooting.md        rev 3  yesterday│
│  api               11          │                                         │
│  troubleshooting    4          │ Health                                  │
│  release-notes      2          │  ○ Document server     240 ms           │
└──────────────────────────────────────────────────────────────────────────┘
```

10.4 Office editor — `temcoDocs.office` (webview + iframe)

Drawn in full at §4.2. Acceptance criteria:

- **OE-1** Opening never marks the tab dirty and never writes a revision.
- **OE-2** Opening a second time with the same revision reuses the same key and joins the same co-editing session.

- **OE-3** With the server unreachable, the surface shows the read-only banner within 2 s and offers Download and History.
- **OE-4** A save produces exactly one new revision; the revision number in the header updates without reloading the tab.
- **OE-5** Closing with unsaved changes prompts once, and *Save* forces a save (§4.5 status 6) before closing.
- **OE-6** Keyboard: `⌘S` triggers `temco.docs.forceSave`; while the iframe has focus, the host must not steal any key except `⌘S` and `⌘W` (§12.3).

10.5 Markdown editor — `temcoDocs.markdown` (webview, Fluent)

TEXT

```
┌─ Commissioning.md ──────────────────── rev 7 · draft — [Preview] [Publish] ┐
│ B I U H1 H2 • 1. " </> 🔗 ▦ —  ≡  (Fluent Toolbar, size=small, 22px)       │
├──────────────────────────────────────────┬──────────────────────────────────┤
│ ## Commissioning the T3000               │  (preview pane, toggleable)      │
│                                          │  Commissioning the T3000         │
│ 1. Connect power to terminals 24 V.      │  ──────────────────────────      │
│ 2. Open the T3000 app and select…        │  1. Connect power…               │
│                                          │                                  │
│ | Input | Range |                        │                                  │
│ |-------|-------|                        │                                  │
│ | AI1   | 0–10V |                        │                                  │
├──────────────────────────────────────────┴──────────────────────────────────┤
│ Ln 12, Col 4 · 1,204 words · Markdown · saved 14:02                         │
└─────────────────────────────────────────────────────────────────────────────┘
```

Rules carried over from the incidents that produced this design:

- **MD-1** The document is never replaced while the editor is focused. Outside changes are queued and applied on blur.
- **MD-2** A message that does not change the text must not touch the document (no caret jump, no undo entry).
- **MD-3** Opening never dirties the tab.
- **MD-4** Paste of an image inserts and saves an attachment asset; paste of HTML is sanitised to the allowed schema.
- **MD-5** The context menu is ours (Fluent `Menu`), so a right-click inside a selection keeps the selection.

10.6 Properties — `temcoWorkbench.properties` (webview, secondary side bar)

TEXT

```
┌─ Properties ──────────────────────────────┐
│ Title      [Commissioning.docx        ]   │
│ Slug       [commissioning             ]   │
│ Area       [guides                  ▾ ]   │
│ Product    [T3000                   ▾ ]   │
│ Format     docx                   (read)  │
│ Revision   7 · 41 KB · 2026-10-23 14:02   │
│ Published  rev 6 → T3000 13.0             │
│ [Show History]  [Download Copy…]          │
└───────────────────────────────────────────┘
```

10.7 Publish — `temcoWorkbench.publish` (webview)

TEXT

```
┌─ Publish T3000 13.0 ──────────────────────────────────────────────────────┐
│ Channel  ( ) Internal  (•) Public         [Publish 3 assets]              │
│                                                                           │
│ ┌─ Check ─────────────────────────────────────────────────────────────┐   │
│ │ ✓ All 3 anchors resolve                                           │   │
│ │ ✓ 1 code block language missing  → warning                        │   │
│ │ ✓ No asset is open in an editor                                   │   │
│ │ ✓ 2 assets already published at this revision (will be skipped)   │   │
│ └─────────────────────────────────────────────────────────────────────┘   │
│                                                                           │
│ Queue: Commissioning.docx · Troubleshooting.md · Release-notes.docx       │
└───────────────────────────────────────────────────────────────────────────┘
```

10.8 Publication queue — `temcoWorkbench.publish` (webview, admin)

TEXT

```
┌──────────────────────┬─────┬────────────┬────────────┬─────────┬─────────┬────────┐
│ Asset                │ Rev │ Channel    │ State      │ Started │ Time    │        │
├──────────────────────┼─────┼────────────┼────────────┼─────────┼─────────┼────────┤
│ Commissioning.docx   │ 7   │ public     │ live       │ 14:02   │ 1.8 s   │ log    │
│ Troubleshooting.md   │ 3   │ public     │ building   │ 14:02   │ 0.4 s   │ log    │
│ Release-notes.docx   │ 9   │ internal   │ failed     │ 13:58   │ 2.1 s   │ retry  │
└──────────────────────┴─────┴────────────┴────────────┴─────────┴─────────┴────────┘
```

Columns: asset · revision · channel · state (`queued | building | live | failed`) · started · duration · actions (*Open log*, *Retry*). Failed rows keep their log and offer Retry; retry re-enqueues the **same** `assetId@revision`, never "latest".

10.9 Releases and Downloads — tree views (register A)

TEXT

```
Releases                                        Downloads
  13.0    2026-09-14  latest  verified            T3000-Setup.exe          48 MB
  12.9    2026-03-02  LTS     shield              T3000-Setup.zip         112 MB
  12.10   2025-11-20  beta    beaker              T3000-Firmware.bin        9 MB
  12.8    2025-06-11  EOL                         T3-ESP-Firmware.bin       2 MB
▸ T3-ESP                                        ▸ SDK
▸ CM5                                           ▸ Documentation packs
```

Rows show version, date, and a stability marker as a codicon (`$(verified)` latest, `$(shield)` LTS, `$(beaker)` beta). Opening a release opens a small Fluent detail view listing its assets; downloading writes through `vscode.workspace.fs` after a save dialog.

10.10 Search — built-in (register A)

TEXT

```
┌──────────────────────────────────────────────────────────────────────────┐
│ asset:docx product:T3000 output                          [Aa] [.*] [ab]  │
├──────────────────────────────────────────────────────────────────────────┤
│ Commissioning.docx  ›  guides  ›  T3000                    4 matches     │
│   12   …connect power to terminals 24 V, then open the…                  │
│   31   …output calibration is described in chapter 4…                    │
│ Troubleshooting.docx  ›  troubleshooting  ›  T3000         1 match       │
│    8   …no output on terminal 5: check the schedule…                     │
├──────────────────────────────────────────────────────────────────────────┤
│ 5 results in 2 assets      filters: asset:docx product:T3000 output      │
└──────────────────────────────────────────────────────────────────────────┘
```

`workbench.view.search` with a contributed `search` context filter: `asset:docx product:T3000`. Do not build a second search UI; extend the one that already has regex, replace, history, and the tree.

10.11 Review — `temcoWorkbench.review` (webview)

```
TEXT
```

```
┌─ Review ──────────────────────────────────────────────────────────────────┐
│ Awaiting review (2)                          diff: rev 6 → rev 7          │
│  ▸ Commissioning.docx   Ada   2 h   [Approve] [Request changes]           │
├──────────────────────────────────────────────────────────────────────────┤
│ Text diff (for md/docx-as-text).  For Office files: open both revisions.  │
└──────────────────────────────────────────────────────────────────────────┘
```

Honest limitation: a visual diff of `.docx` inside our own UI is out of scope for phase 1. The surface offers *Open rev 6* and *Open rev 7* side by side is forbidden (one editor column), so it offers **Open rev 6**, then **Open rev 7**, and records which the reviewer saw.

10.12 Diagnostics — `temcoWorkbench.admin` (webview)

A health matrix, the single place to debug a broken document server:

```
TEXT
```

```
┌─ Diagnostics ─────────────────────────────────────────────────────────────┐
│ Server URL     https://docs.temco.local                      (configured) │
│ JWT secret     •••••••• set 3 days ago                        (ok)        │
│ Health check   200 in 240 ms                                  (ok)        │
│ Version        ONLYOFFICE Docs 9.0.4                                      │
│ Reachability   platform → server ✓      server → platform ✓               │
│ Last error     none                                                       │
│ [Run health check]  [Edit secret]  [Copy diagnostics JSON]                │
└──────────────────────────────────────────────────────────────────────────┘
```

Reachability runs a real probe: the platform asks the server to fetch a fixture URL (`/api/docs/ping/<token>`) and reports whether it saw the hit. This single row diagnoses T1, the failure that costs hours.

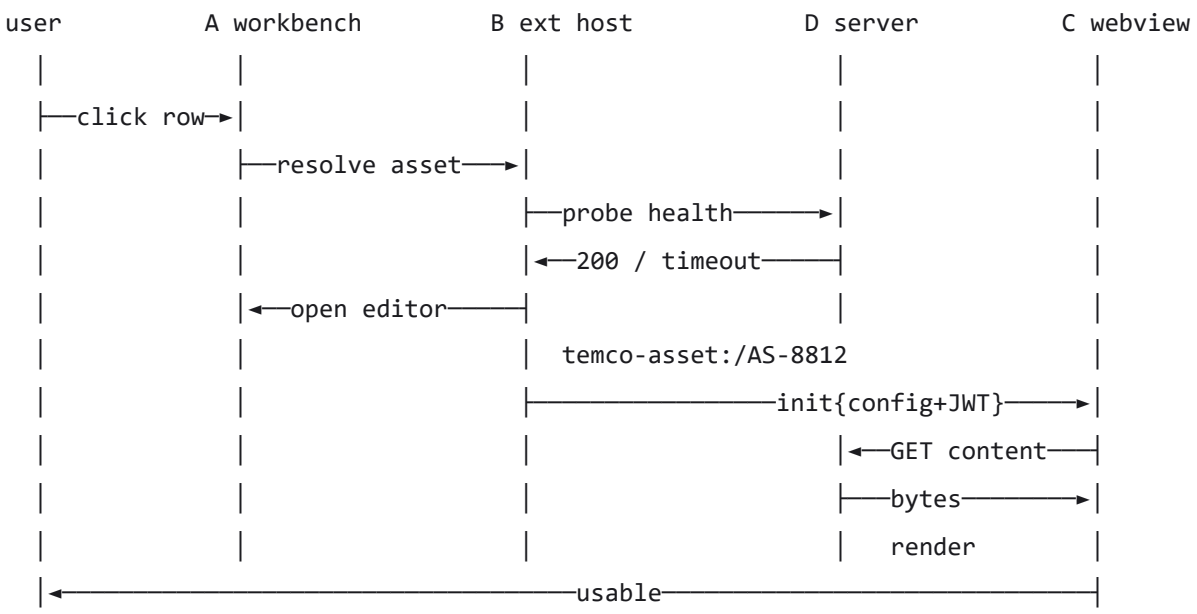
---

11. Flows

Each flow names its participants by plane: **A** workbench, **B** extension host, **C** webview, **D** document server, **E** store.

F1 — Open a Word document from the tree (the primary flow)

TEXT

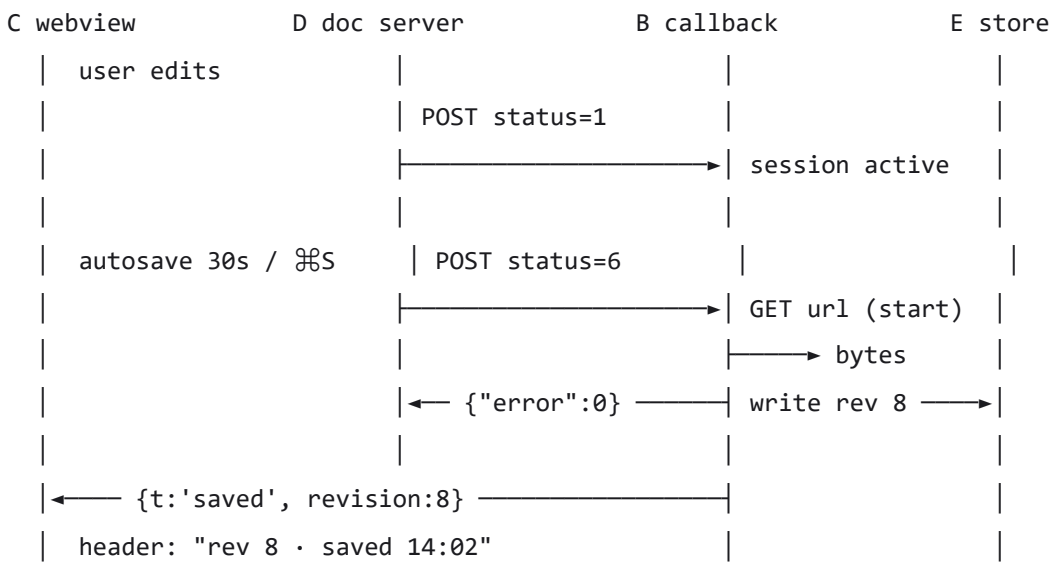


1. **click row** — the `TreeDataProvider` resolves the element to an `AssetId`; no network yet.
2. **resolve asset** — read the `AssetRecord` from the store; `format: docx` selects the `office` host (§4.11).
3. **probe health** — `GET /healthcheck`, 2 s timeout, cached 30 s. On timeout jump to F5.
4. **open editor** — `vscode.commands.executeCommand('vscode.openWith', temcoAssetUri, 'temcoDocs.office')`.
5. **temco-asset:/AS-8812** — the tab opens immediately with a skeleton, before any network.
6. **init{config+JWT}** — the host mints a single-use ticket (§4.4), builds and **signs** the config (§4.7), and posts it to the webview.
7. **GET content** — the document server fetches the signed URL **itself**. The client never sends the bytes.
8. **bytes** — streamed from our content handler; range requests are supported.
9. **render** — `onDocumentReady` → the webview posts `{t:'ready'}` → the host clears the skeleton.
10. **usable** — the user can type; focus is now owned by the iframe (§12.2).

Failure branches: health timeout → banner, no iframe (§4.10). Content 403 because the ticket expired → `{t:'error', code:'content-expired'}` → one automatic retry with a fresh ticket, then banner.

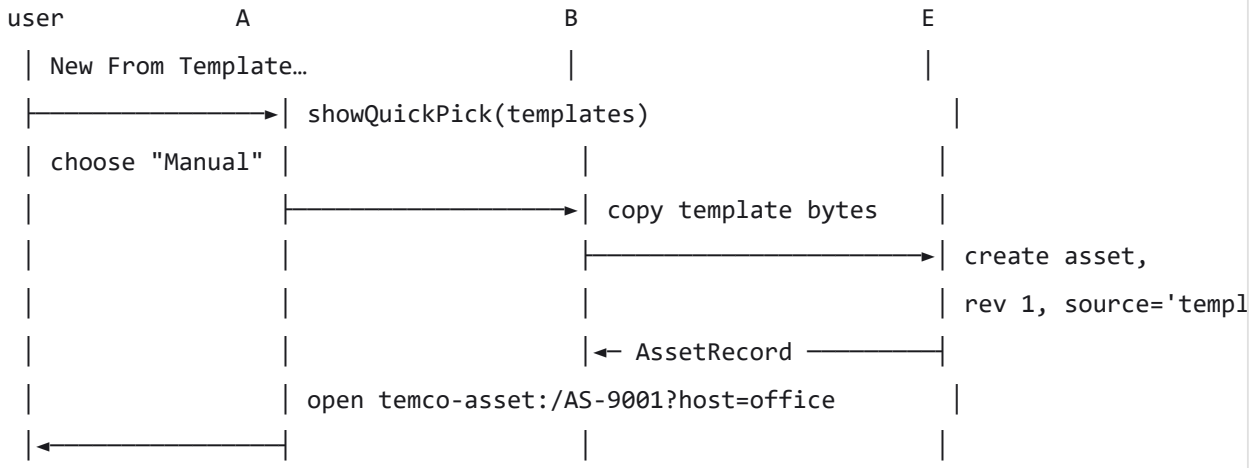
F2 — Edit, force save, revision bump

TEXT



F3 — New document from a template

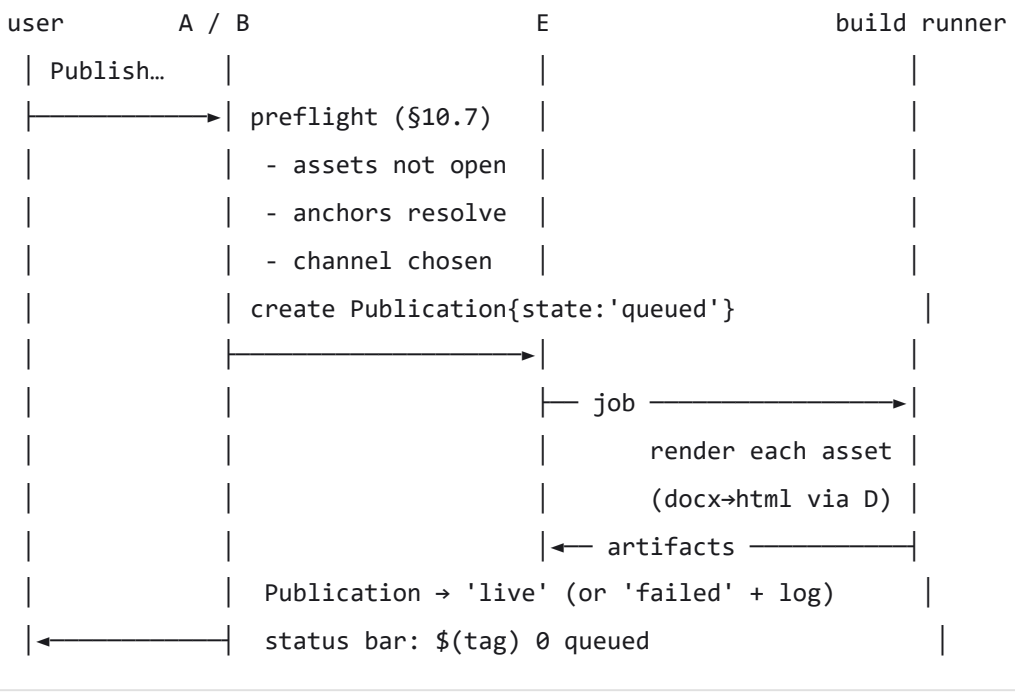
TEXT



Why copy instead of "Save As": ONLYOFFICE has no template API. A template is a plain asset marked `kind: 'template'`; instantiating is a byte copy plus a new asset row. This also means templates are versioned like everything else.

F4 — Publish a product version

TEXT



Publications are **frozen**: they store `assetId@revision` tuples. Publishing never reads "latest" at build time.

F5 — Document server unreachable

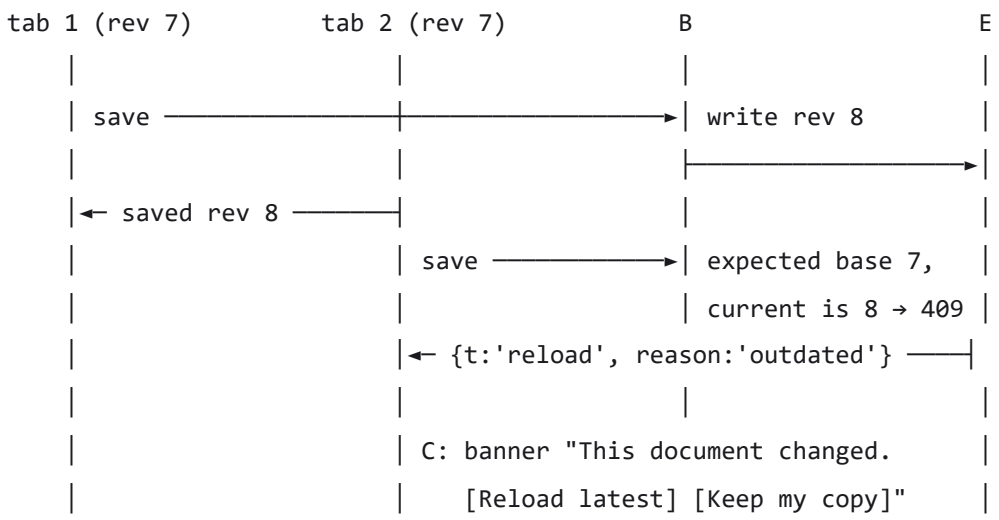
TEXT



Never: a blank iframe, an infinite spinner, or a silent fallback to the text editor (that would let a user edit a `.docx` as bytes and destroy it).

F6 — Two editors, one asset, revision conflict

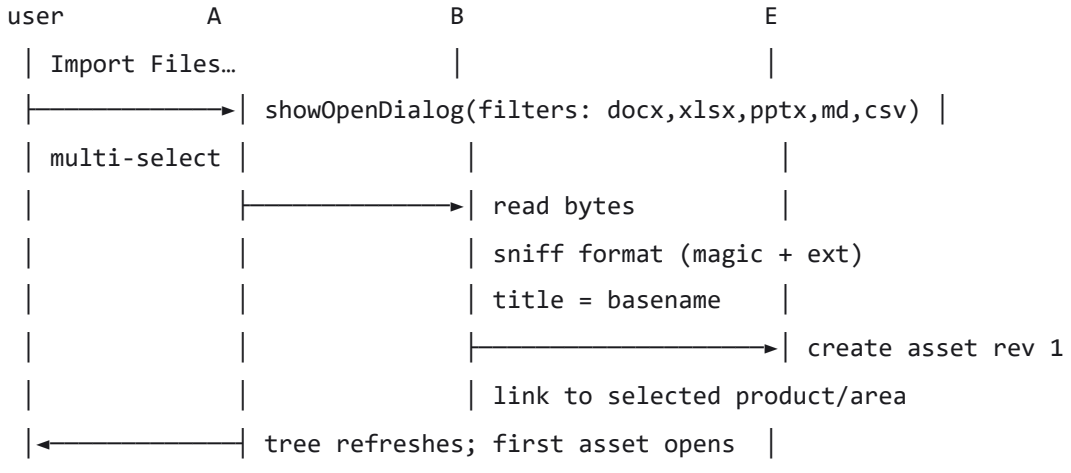
TEXT



Both revisions survive (D1, D2). "Keep my copy" writes the user's bytes as rev 9 with `message: 'from conflicting edit'`, so nothing is lost.

F7 — Import files

TEXT



F8 — History and restore

TEXT

```
context menu → Show History
     │
     ▼
┌─ History: Commissioning.docx ──────────────────────────┐
│ rev 7  41 KB  14:02  editor   • current                │
│ rev 6  41 KB  09-14  editor                            │
│ rev 5  39 KB  09-02  import                            │
│              [Open]  [Restore as rev 8]  [Download]    │
└────────────────────────────────────────────────────────┘
```

Restore never rewinds; it copies old bytes forward as a new revision (D2).

F9 — Open With (forcing a host)

TEXT

```
context menu → Open With…  →  QuickPick, ordered by the registry
     │
     ▼
┌──────────────────────────────────────┐
│ Office (Word)          • default     │
│ Markdown editor                      │
│ Text Editor                          │
└──────────────────────────────────────┘
     │ choose "Text Editor"
     ▼
temco-asset:/AS-8812?host=text  →  opens in the native text editor (read/write)
```

This is the escape hatch when a document is corrupt, and the mechanism the user asked for when they said *"when you choose … at the default md editor to change*".

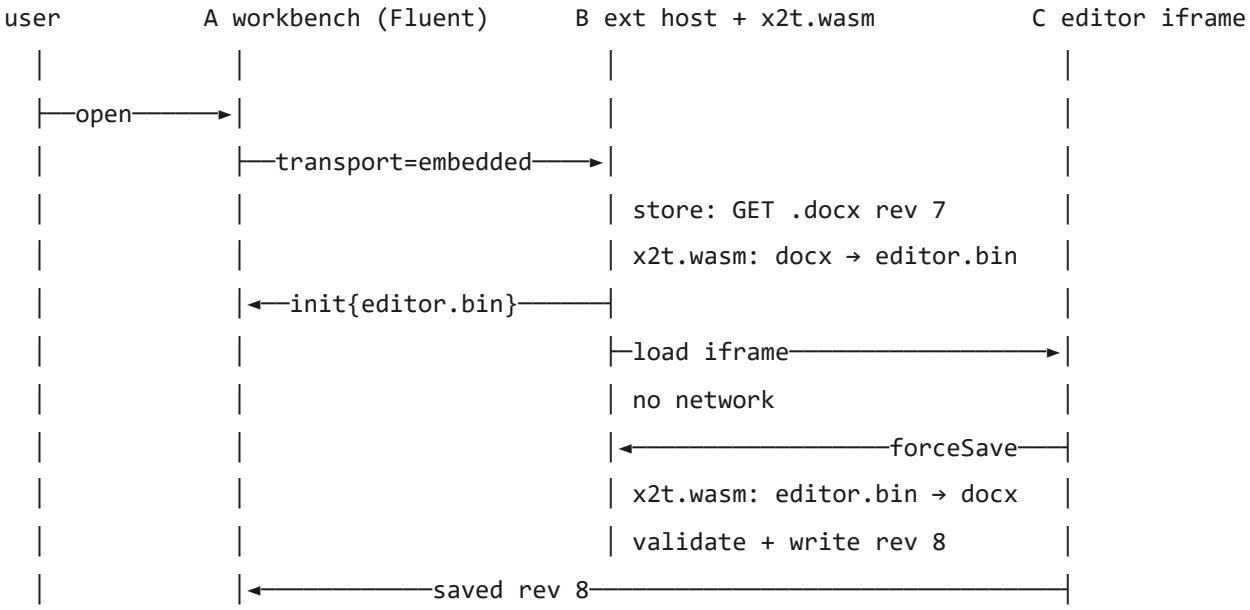
F10 — Search to open

TEXT

```
⌘K → type "commissioning"            (command center, register A)
     │
     ▼
results: assets · commands · settings · files
     │ Enter on an asset
     ▼
temco.asset.open → F1
```

F11 — Open and save with no server (Topology B)

TEXT



Same host protocol as F1/F2, so nothing above the transport changes. The differences are all in `B`: conversion runs in a Worker (~0.4–2 s depending on document size), there is no health probe, no ticket, no signed URL, and no callback POST. `forceSave` therefore means "convert now" rather than "ask the server to flush".

---

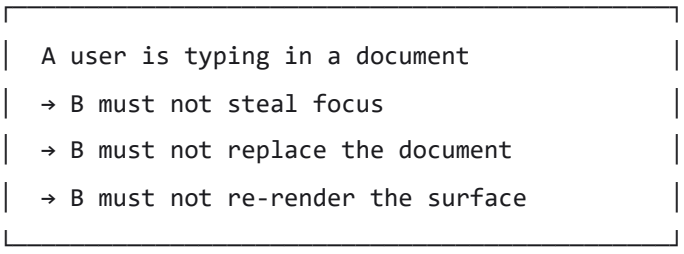
12. Keyboard, focus, accessibility

12.1 Shortcut ownership

Scope	Owner	Examples
Global chords	A (VS Code)	`⌘K`, `⌘⇧P`, `⌘⇧E`, `⌘W`, `⌘,`
Asset actions	A	`⌘S` → `temco.docs.forceSave`, `F2` rename, `⌘⇧O` open with
Inside our webview	B/Fluent	`⌘F` find-in-asset, `Esc` close popover, `Tab` traversal
Inside the document iframe	D (ONLYOFFICE)	Everything, including `⌘B`, `⌘F`, `Enter`

12.2 The focus contract (the rule that prevents the "typing goes somewhere else" class of bug)

```
TEXT
```



Implementation rules:

1. **FR-1** Any host→content update is queued while the content surface reports `focused: true`, and applied on `blur` (§10.5 MD-1).
2. **FR-2** An update that does not change content is dropped, not applied (§10.5 MD-2).
3. **FR-3** Never call `webview.postMessage` on a timer while the user is typing, except the save heartbeat, which carries no document bytes.
4. **FR-4** Never re-create the iframe element to "refresh" it. Change its `key` (React) only when the ONLYOFFICE key changes (§4.3).
5. **FR-5** When the iframe has focus, the host registers no `keydown` capture handler except for the allowlist in §12.3.

12.3 Routing `⌘S` while the iframe has focus

A cross-origin iframe swallows key events. VS Code's `keybindings` still fire because they are evaluated in the host document **before** the webview, but a keybinding that only makes sense inside our editor

must be scoped:

```jsonc
{
        "command": "temco.docs.forceSave",
        "key": "ctrl+s",
        "mac": "cmd+s",
        "when": "activeCustomEditorId == 'temcoDocs.office' || activeCustomEditorId ==
}
```

Track iframe focus in the webview (`window.blur` + `document.activeElement === iframe`) and post `{t:'focus', focused}` so the host can enable iframe-only commands (e.g. `Enter` still belongs to the document, so it is never claimed).

12.4 Accessibility requirements

Requirement	Implementation
Every Fluent control has a label	`Field label={…}`; no placeholder-only inputs
The header and status bar are reachable by keyboard	Plain DOM order; skip-link to the canvas
The iframe has a title	`title="Document editor: Commissioning.docx"`
Server state is announced	`MessageBar` in an `aria-live="polite"` region, once per state change
Colour is never the only signal	Save state uses icon + text (`● unsaved` / `✓ saved 14:02`)
High contrast	Bridge maps `vscode-high-contrast` → ONLYOFFICE `theme-contrast-dark`; Fluent theme built with `createHighContrastTheme`
Focus visible in our chrome	Fluent's `:focus-visible` outline, never removed
Screen reader can tell registers apart	The webview root gets `aria-label="Document content"`, the side bar is native

13. Performance budgets

Budget	Target	Measured where
Tree first paint (warm)	< 300 ms	`performance.mark` in the provider
Any tree expansion (300 children)	< 100 ms	provider timer
Webview boot (React + Fluent, cold)	< 400 ms	`ready` message timestamp
Office editor: server probe	< 2 000 ms timeout, cached 30 s	health client
Office editor: open → `onDocumentReady` (warm)	< 4 000 ms	`onDocumentReady` event
Office editor: open → ready (cold server)	< 9 000 ms	same
Save callback: GET started	< 500 ms	callback handler
Save callback: revision visible in UI	< 1 500 ms	`saved` message
Webview bundle (each surface, gzip)	< 350 KB	build report
Fluent import: one copy only	verified by build	bundle analysis

Cold-start is dominated by the document server, not by us. Pre-warming on extension activation is allowed **only** after 3 s idle and **only** the `/healthcheck` probe — never a document load.

---

14. Testing

14.1 The pyramid, adapted to a two-register product

Level	What	Tool	Runs
Unit	key derivation, config builder, JWT signing, callback validation, snapshot rules, tree derivation	`node --test` against compiled `out/`	every commit
Contract	callback handler against recorded ONLYOFFICE payloads (status 1/2/3/4/6/7)	in-process HTTP	every commit
Webview render	each surface mounts, no throw, key elements present	jsdom (`check-webview.mts`)	every commit
Real-browser editing	typing, caret, selection, paste, undo on the Markdown editor	Playwright (`check-editor.mts`)	every commit
End-to-end	open a real `.docx` in a real VS Code window against a real document server	`validate-ui-scenario` scenario	nightly / on demand
Accessibility	keyboard-only pass, high contrast, screen-reader labels	manual checklist + axe on webviews	before each phase exit

14.2 Non-negotiable regression tests

1. **Key test** — same `(assetId, revision)` ⇒ same key; revision bump ⇒ different key; key matches `/^[A-Za-z0-9_-]{1,128}$/` .
2. **Round-trip law** — open then close with no edit ⇒ zero new revisions, tab not dirty. Applies to both Office (status 4) and Markdown.
3. **Callback replay** — a second POST with the same ticket is rejected and does not write.
4. **Truncated body** — a callback whose GET returns HTML instead of a zip is rejected, no revision written, error surfaced.
5. **Caret** — for the Markdown editor, a host `set` that does not change text must leave `selection.head` untouched. (This test exists and has caught the bug before.)
6. **Degradation** — with the probe forced to timeout, the Office surface renders the banner and no iframe within 2 s.
7. **Dual-host markdown** — the same `.md` fixture opened through the Fluent host and through the Office host each satisfies the round-trip law (open, close, zero revisions), and `preferredHost` survives a reload (§4.8.1 H1, H2).
8. **Host fallback** — an asset whose `preferredHost` cannot serve its format still opens, with exactly one warning logged (H4).

9. **Licence gate** — a build with `TEMCO_BUILD_CHANNEL=public` and the office pack present but `TEMCO_OFFICE_LICENCE` unset **fails**, and the same build with `commercial` passes (LG-1).
10. **Pack missing** — with the pack removed, Office assets open read-only with the banner, no crash, and no silent text-editor fallback (LG-4). `x2t.wasm` / `web-apps` version mismatch reports `office-pack-mismatch` (LG-5).

14.3 What cannot be tested in CI

The ONLYOFFICE canvas is cross-origin. We cannot assert on its DOM. We assert on **our** state machine (config sent, callback received, revision written) and screenshot the canvas for human review. Do not write a test that reaches into the iframe.

---

15. Security

Risk	Control
Content URL reused by someone else	Single-use ticket + HMAC + 15 min expiry; bound to `(assetId, revision)`
JWT secret leak	`SecretStorage` only; never a setting, never logged, never sent to the webview
Malicious document	Documents are data, not code — but the *editor* runs them in a sandboxed origin. Never render a document in our own origin
SSRF via content URL	The URL host must equal the configured platform origin; reject anything else
Callback spoofing	Per-session ticket in the callback URL; unknown tickets are 404 + logged
Oversized save	`temco.docs.maxRevisionBytes` enforced before read; stream with a cap
Path traversal in asset ids	`assetId` matches `/^AS-[0-9]{4,8}$/`; never used as a filename
Iframe framing our origins	`frame-src` limited to `${officeOrigin}`; `X-Frame-Options` on the platform API
Clipboard of a pasted image	Written through the extension host into the asset store as a new asset (§10.5 MD-4), never as a data URL in the document

---

16. Repository shape and dependencies

TEXT

```
temco-platform/                     (VS Code 1.144.0.0 fork)
  src/vs/…                          untouched
  extensions/
    temco-ui/                       ★ NEW: the design system
      src/index.ts
      src/provider/TemcoProvider.tsx
      src/theme/readVSCodeTheme.ts
      src/theme/toFluentTheme.ts
      src/density/compact.ts
      src/components/…
      package.json                  deps: @fluentui/react-components, @fluentui/react-ic
    temco-docs/                     the docs module
      src/
        asset/store.ts              revisions, invariants D1–D5
        asset/tree.ts               TreeDataProvider
        office/config.ts            buildOfficeConfig, documentKey, jwt
        office/client.ts            health probe, version probe
        office/callback.ts          the HTTP listener (§4.5)
        office/content.ts           signed streaming handler (§4.4)
        office/transport/http.ts    OfficeTransport — server-hosted (§4.1.1 A)
        office/transport/embedded.ts OfficeTransport — x2t.wasm in-process (§4.1.1 B)
        office/transport/index.ts   choose by config; capability report
        hosts/registry.ts           EditorHost table (§4.11)
        publish/state.ts            Publication machine
        commands/*.ts               25 commands (§9.2)
      webview/
        office/                     React host: header, banner, status, iframe
        markdown/                   React + Tiptap, Fluent chrome
      package.json                  contributes: customEditors, commands, settings, view
    temco-workbench/                shell contributions (exists: 4 views, 37 commands)
    temco-projects/ temco-repositories/ temco-store/ temco-ai/ …   (existing)
```

Boundary lint (add to `eslint`):

Rule	Enforced
`@fluentui/*` importable only from `extensions/temco-ui/src/**`	`no-restricted-imports` with `overrides`
`temco-ui` must not import `vscode`	keeps it renderable in tests and in a browser
`temco-docs/src/**` must not import React or Fluent	the extension host has no DOM
`temco-docs/webview/**` must not import `vscode`	webviews only speak `postMessage`
No `document.` in `src/**`, no `require('fs')` in `webview/**`	same boundary, caught early

17. Roadmap

Phase	Ships	Exit criterion
P1 — Office spine	`temco-ui` + Fluent bridge; `temcoDocs.office` custom editor on the **embedded** transport; office pack + licence gate (§4.1.5); asset store with revisions; product/documentation trees; Open With	A real `.docx` opens, edits, saves, and reopens in the real VS Code window **with the network off**; round-trip law passes; a public-channel build without a licence fails CI
P2 — Authoring	Markdown editor on Fluent; properties panel; history/restore; import; templates	An author writes a whole guide set without leaving the platform
P3 — Publish	Publication machine, preflight, queue, DocFX build, downloads	Publishing T3000 13.0 produces a site and a zip from frozen revisions
P4 — Breadth	Excel/powerpoint surfaces; diagram host; review flow; search filters; **`http` transport for co-editing**	Every asset kind has exactly one host and one acceptance test; co-editing works when a server is configured
P5 — Knowledge	Chunking, embeddings, knowledge edges, AI Q&A over published text	"How does Output 5 work?" answers with citations to revisions

18. Definition of done, and open decisions

18.1 Done means

- [ ] Every §10 screen has its acceptance criteria met, checked in a real VS Code window.

- [ ] Every §14.2 regression test exists and fails when its fix is removed.
- [ ] No Fluent import outside `temco-ui`; the lint rule is enabled and green.
- [ ] No `--vscode-*` variable is used to colour a Fluent surface directly (the bridge does it).
- [ ] Document server down produces §4.10 states, verified by hand.
- [ ] The extension packs and installs as a VSIX; the document server address is configured, never bundled.
- [ ] `docs/temco-platform/modules/temco-docs/extension-design.md` updated with what was actually built.

18.2 Decisions still open (recommendation in brackets)

1. **Transport** — **decided:** support both, `embedded` by default, `http` opt-in (§4.1.4). Co-editing is therefore *not* in the default build, and the UI must say so (TR-4).
2. **Distribution scope** — **decided:** internal now, may be sold later. This is exactly the case that needs the build gate in §4.1.5, because the AGPLv3 question only becomes urgent at the moment of conveyance. Next action: legal confirm of LG-1's wording before P1 ships, not before it starts.
3. **Document server edition** — Community (free, concurrency limits, no white-label) or commercial. This decides whether the ONLYOFFICE logo appears in our editor. [Start Community; budget for commercial when branding matters.]
4. **Where the document server runs** — one shared intranet host, or `localhost` Docker on every machine (§4.9). Shared is recommended: co-editing and shared review only exist with a shared server, and per-machine Docker is more setup per person, not less. [Shared intranet host; `localhost` for development only.]
5. **Where the asset store lives** — local SQLite for a single workstation, or a shared Postgres for a team. The content and callback endpoints imply a reachable host, which argues for shared. [Shared Postgres + a small HTTP service; SQLite only for a personal profile.]
6. **Markdown host** — **decided:** support both, remembered per asset (§4.8.1), Fluent by default. Remaining sub-question: whether `temco.docs.markdown.engine` should exist at all now that `preferredHost` covers it. [Keep it; it is the default for assets that have no explicit choice.]
7. **Whether `.md` and `.docx` may hold the same page** — i.e. one asset, two formats. [No. One asset, one format. A conversion creates a new asset with a `derived-from` edge.]
8. **Engine fallback if AGPL blocks distribution** — four routes in §4.1.7: R1 separate service, R2 commercial licence, R3 MPL engine (ZetaOffice), R4 no engine at all. [Start on **A + B + R4** (author in Fluent, view via `docx-preview`, convert-on-import with a warning) because that ships with zero AGPL exposure; add **R1** the day a host exists. Re-evaluate at the point the product is sold.]
9. **How much existing `.docx` must be edited in place** — this, not preference, decides between R4 and R1/R2/R3 (§4.1.7.1). [Answer it with a corpus count: how many documents, and how many of them are round-tripped weekly.]

18.3 One-paragraph brief for the generator

The brief is §23, at the end of this document, so it can name the feature under construction.

> Build TEMCO Platform inside the existing VS Code fork. The shell is VS Code and must not be touched; every screen we own is a webview built with React 19 and Fluent UI v9 from `extensions/temco-ui`, themed by bridging `--vscode-*` variables into a Fluent theme. Office documents are edited by ONLYOFFICE, behind a pluggable `OfficeTransport`: either a Document Server embedded as a single cross-origin iframe (JWT-signed config, `key` derived from `(assetId, revision, options)`, short-lived signed content URL, save callback that writes an immutable revision), or a fully embedded build with static `web-apps` plus `x2t.wasm` where conversion happens in-process and no server exists at all. Markdown has two hosts — a Fluent editor by default, the same Office host when the asset remembers that choice in `preferredHost` — resolved by `explicit override → preferredHost → setting → format priority`. The domain is five concepts — Product, Asset, Editor Host, Publication, Knowledge — and no sixth noun may appear. Every save is a revision; nothing edits in place; nothing ever shows a blank iframe.

---

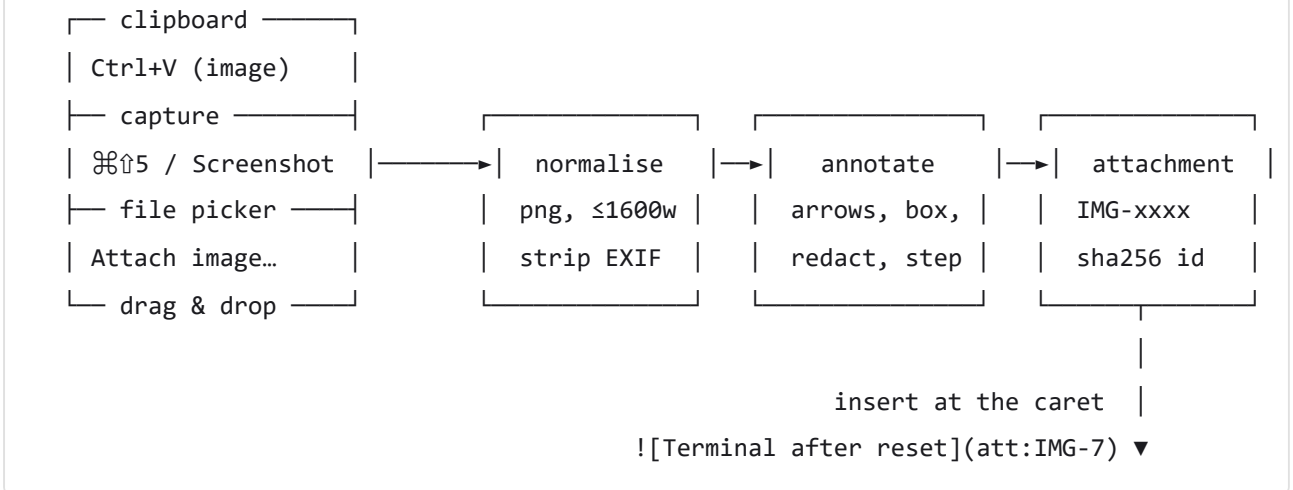
22. The document feature — the whole flow

Scope decision: **this is the only feature we build now.** Products, hardware, firmware and releases stay as data the document feature can reference; their screens come later. Everything below is needed for an author to sit down and write a page with screenshots in it, and for the page to stay correct across versions.

22.1 One pipeline for every image

Three entrances, one path. The author never picks a format, a folder, or a filename.

TEXT



Rules that make this worth building:

- **P1** An image is never a data URL in the document. It becomes an attachment with an id, and the document holds `att:IMG-7`. A 400 KB base64 blob inside a markdown file is how a docs repo becomes uncloneable.
- **P2** Capture, paste and file all end in the same annotation surface — the author does not get a worse experience because of where the bitmap came from.
- **P3** The attachment id is the **content hash**, so pasting the same screenshot twice reuses one attachment and one file on disk.
- **P4** Insertion is at the caret, never appended. The editor's selection survives, which is the whole point of "quick".
- **P5** Failure is visible: no clipboard permission, no capture permission, unsupported type — each gets a sentence naming the cause and the fallback.

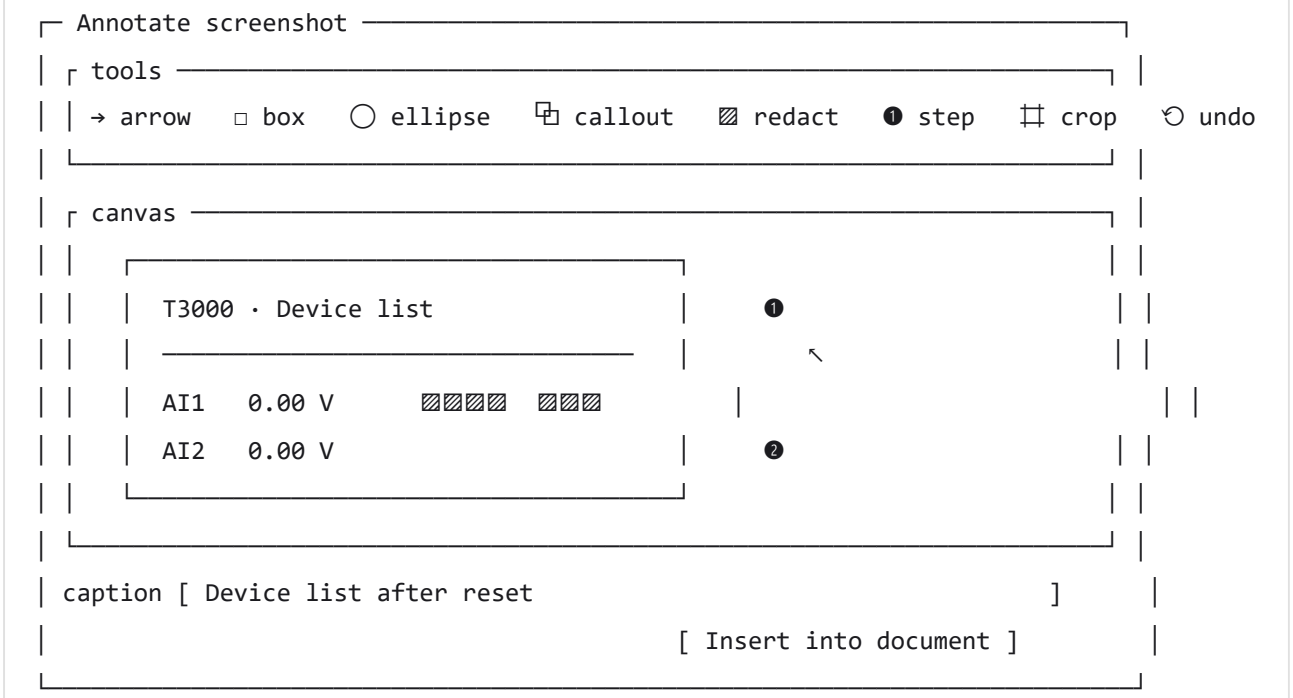
22.2 The paste matrix — what each clipboard flavour becomes

Clipboard holds	Result	Why
PNG / JPEG / DIB bitmap	attachment + `![…](att:IMG-n)`	the common case: a screenshot of the T3000 app
HTML with a `<table>`	a markdown table, cells flattened	pasting a table from Excel must stay a table
HTML with `<ul>/<ol>`	a markdown list	
HTML with headings/paragraphs/bold	markdown with those marks kept, colours and fonts dropped	we do not carry Word's styling into markdown
HTML with `<img src>` that is reachable	downloaded and re-hosted as an attachment	a link to someone's intranet breaks in a year
Plain text	plain text, at the caret	
Files (`CF_HDROP`)	each file attached; images go through §22.1	
A URL on its own	offered as a link, not fetched	

The one rule behind the table: **format follows the source's structure, not its styling.**

22.3 The annotation surface

TEXT



- **A1** Every tool works with one drag. No property panels, no selection handles.
- **A2 Redact is not blur.** Blur is shown as a convenience, but the redact tool *replaces* pixels with a solid block and the UI says so, because a blurred licence key is a published licence key.
- **A3** Undo undoes one operation, not the session. The author keeps working after a mistake.
- **A4** Annotations are baked into the PNG on insert. Nothing downstream has to understand them, and a reader cannot remove them.
- **A5** The caption becomes the `alt` text — accessibility and search for free, and it stops "image1.png" reaching a customer.
- **A6 Callout** magnifies the dragged region (≈2.2×) into an inset connected by leader lines, so a small terminal value can be read on a 900 px screenshot. It samples the pixels underneath, so an arrow drawn first also appears magnified.
- **A7 Crop** bakes everything drawn so far and is undoable once. After a crop the canvas is the new truth; the author is never asked to confirm a destructive step they can reverse.

22.4 Attachments age; documents do not

A screenshot is a photograph of a version. When firmware 13.1 ships, every screenshot of the 13.0 device list is a lie that still looks true.

TEXT

```
attachment IMG-7
├── data         sha256 → stored once
├── caption      "Device list after reset"
├── appliesTo    "13.0"            ← what the picture shows
├── referencedBy AS-8812 rev 7     ← who points at it
└── state        fresh | stale | orphan
```

- **S1** `appliesTo` is set when the image is inserted (the document's version at that moment), and it is editable.
- **S2** A version change (a release, or the automation in §22.5) marks every attachment whose `appliesTo` is the old version **stale**, in the document and in a review queue.
- **S3** Stale is a warning on the page, not a block. Publishing a stale screenshot is allowed and recorded.
- **S4** An attachment referenced by nothing is shown as orphaned rather than deleted silently.

22.5 Automation: an agent proposes, a person approves

The stakeholder ask is "MCP / GitHub actions generate changes automatically as we do new versions". The honest shape of that is a **proposal loop**, not a write-through.

TEXT

```
firmware tag v13.1 pushed
       │
       ▼
GitHub Action: docs-drift
       │  1. run the docs tooling in CI
       │  2. ask the MCP server for the affected assets
       ▼
┌──────────────────────────── MCP tools ────────────────────────────┐
│ docs.find_version_refs(from, to)      → assets + line hits        │
│ docs.get_asset(id)                    → current revision + body   │
│ docs.propose_revision(id, patch, why) → a draft revision, not a save │
│ docs.attach_image(id, png, caption)   → attachment + reference     │
│ docs.list_stale_attachments(version)  → screenshots that lie      │
│ release.notes_from_commits(range)     → a draft release note      │
└───────────────────────────────────────────────────────────────────┘
       │
       ▼
Pull request (or a review item in the app) containing:
  · proposed revisions with a diff
  · the reason ("13.0 appears 7 times; 3 are firmware gates")
  · stale screenshots listed
       │
       ▼
a person approves  →  publication  (never the agent)
```

- **G1** Automation **never** writes to a published revision and never publishes. It proposes.
- **G2** Every proposal carries its evidence: the commit range, the matching lines, the MCP call that produced it.
- **G3** The same loop runs without CI — the editor has an "Automation" surface that shows the proposals and applies them as ordinary revisions, so the feature is useful on day one.
- **G4** An agent may attach an image (a generated diagram, a re-captured screenshot) only through `docs.attach_image`, which applies the same size and format rules as §22.1.

Shipped: `.github/workflows/docs-propose.yml`. It runs on any `v*` tag (or by hand), derives the version pair from the tag, scans the configured folder for the old version, uploads a report and a markdown summary, applies the replacements on a branch and opens a **draft** pull request. It has no path that writes to the default branch, and no path that publishes. The scanner is inline in the workflow because this repository forbids new `.js` files — a scanner outside the tree is a scanner that rots silently.

22.6 What "a very good editor" means here, concretely

Not a feature list — a set of properties that can be tested:

Property	Test
Typing never loses the caret	type 20 characters in the middle of a paragraph; selection is where the author left it
Paste is one keystroke	a screenshot goes from clipboard to annotated and inserted without a dialog chain longer than one
Save is one keystroke	`⌘S` writes a revision; the tab never shows a dirty dot it cannot clear
Nothing is lost by opening	open, close, zero revisions
Every image has words	no attachment reaches a publication with an empty `alt`
The reader can tell truth from history	a stale screenshot is labelled where it is read, not only in an admin screen
A callout shows what it claims	drag over a region; the inset is that region magnified, not a stock box
Nothing irreversible is one click away	crop is undoable; nothing is deleted, only superseded

22.7 What is built

§22 is no longer only a design. The editor it describes ships as the *Temco Markdown* surface of `extensions/temco-docs` — a custom text editor on Fluent UI v9, default for `*.md`, with capture, annotation and the paste matrix in the surface and the file still the document. The implementation record — the protocol, the caret rules, the defects that only a real window showed, and the deliberate deviations from this section — is `modules/temco-docs/extension-design.md`, and the real-window evidence is the `temco-markdown-editor` scenario (nine steps, all passing).

Of this section's rules, the ones that now ship:

Rule	Where it lives
§22.1 one pipeline for every image	`src/attachments.ts` — one landing path for a paste, a capture and a file picked from disk
§22.2 the paste matrix	the surface: an image opens the annotator, HTML tables and lists become Markdown, plain text stays text
§22.3 the annotation surface	`webview/docs/annotate.tsx` — arrow, box, ellipse, callout, redact, blur, numbered step, crop, undo
§22.4 S1 `appliesTo` at insert time	recorded from the release workflow's `version-report.json`
§22.4 S2 a version change marks pictures stale	`staleImages()` over the record, reported by `temcoDocs.staleScreenshots`
§22.4 S3 stale is a warning, not a block	the report records; nothing refuses a publish
§22.6 typing never loses the caret	the three rules in `main.tsx`, asserted by `check-docs` and by the scenario
§22.6 the editor a person actually writes in	outline, find and replace, link to a page, page properties, list continuation, indent, colour behind the words, a draggable split and three view modes — the features the text editor had and the default editor did not

Three parts of this section remain design only: the attachment **state** machine (fresh / stale / orphan), the publish **preflight** that reads the record inside the build rather than beside it, and §22.5's proposal loop, which is shipped as a workflow (`.github/workflows/docs-propose.yml`) but has not yet run against a real release.

---

23. One-paragraph brief for the generator

> Build TEMCO Platform inside the existing VS Code fork. The shell is VS Code and must not be touched; every screen we own is a webview built with React 19 and Fluent UI v9 from `extensions/temco-ui`, themed by bridging `--vscode-*` variables into a Fluent theme. Office documents are edited by ONLYOFFICE, behind a pluggable `OfficeTransport`: either a Document Server embedded as a single cross-origin iframe (JWT-signed config, `key` derived from `(assetId, revision, options)`, short-lived signed content URL, save callback that writes an immutable revision), or a fully embedded build with static `web-apps` plus `x2t.wasm` where conversion happens in-process and no server exists at all. Markdown has two hosts — a Fluent editor by default, the same Office host when the asset remembers that choice in `preferredHost` — resolved by `explicit override → preferredHost → setting → format priority`. **The feature under construction is documents (§22): a markdown editor where a screenshot goes from clipboard to annotated and inserted in one keystroke, every image is an attachment with a caption and a version it depicts, and version changes arrive as proposals from CI that a person approves.** The domain is five concepts — Product, Asset, Editor Host, Publication, Knowledge — and no sixth noun may appear. Every save is a revision; nothing edits in place; nothing ever shows a blank iframe.